LẬP TRÌNH MẠNG

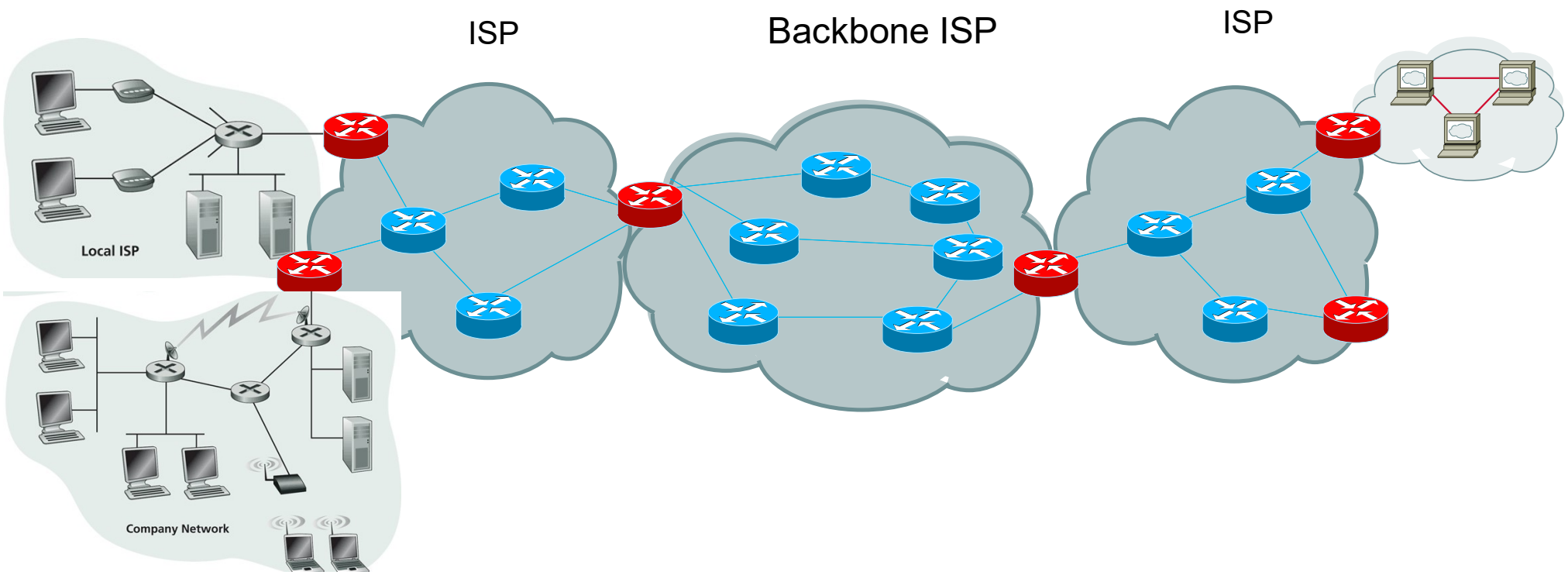
Name: Ths. Tống Thanh Văn
Phone: 0906762900
Mail: tongthanhvan@tdtu.edu.vn

Các khái niệm cơ bản về mạng

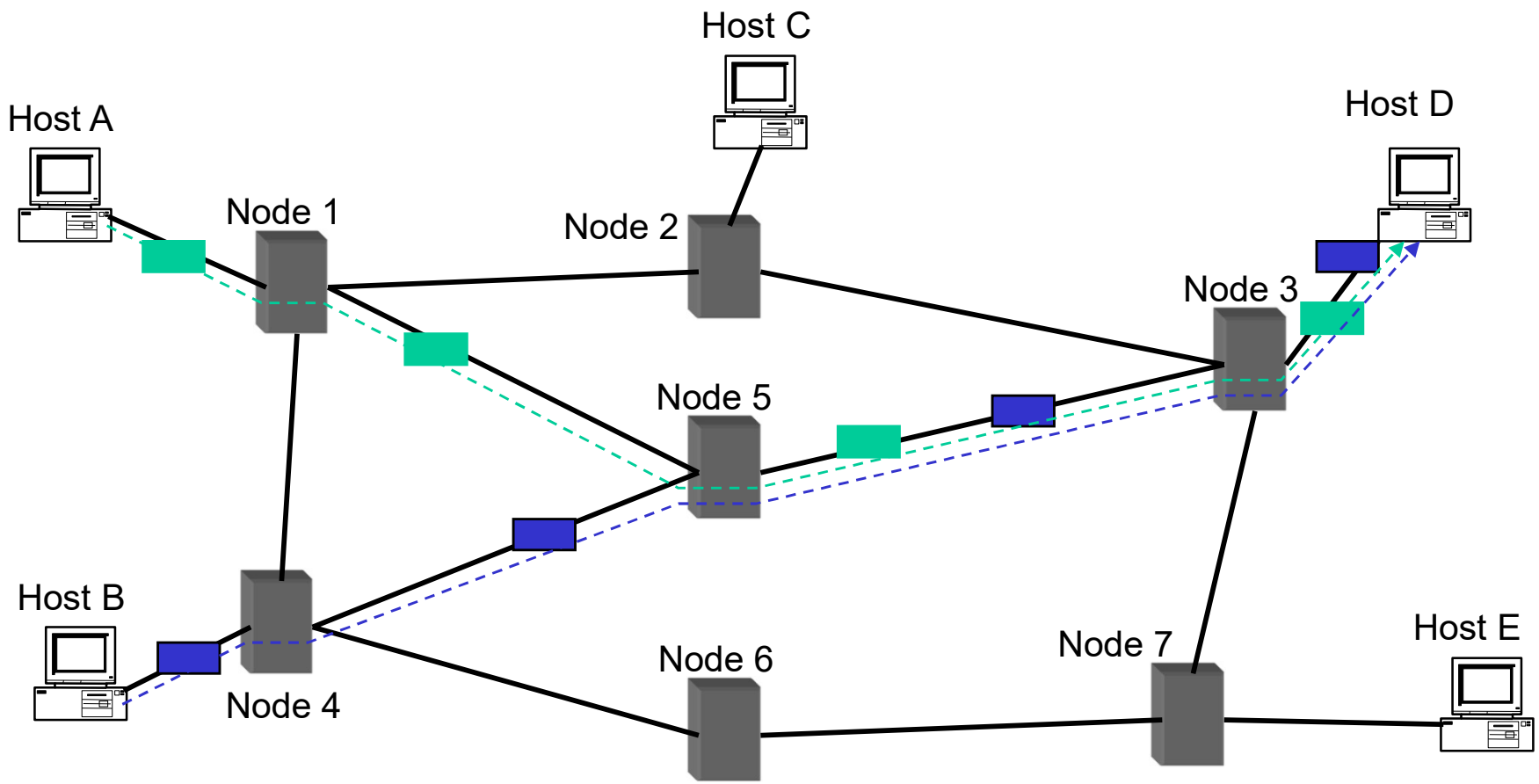
Nội dung

- Mạng máy tính
- Kiến trúc phần tầng của mạng
- Các ứng dụng Client/Server

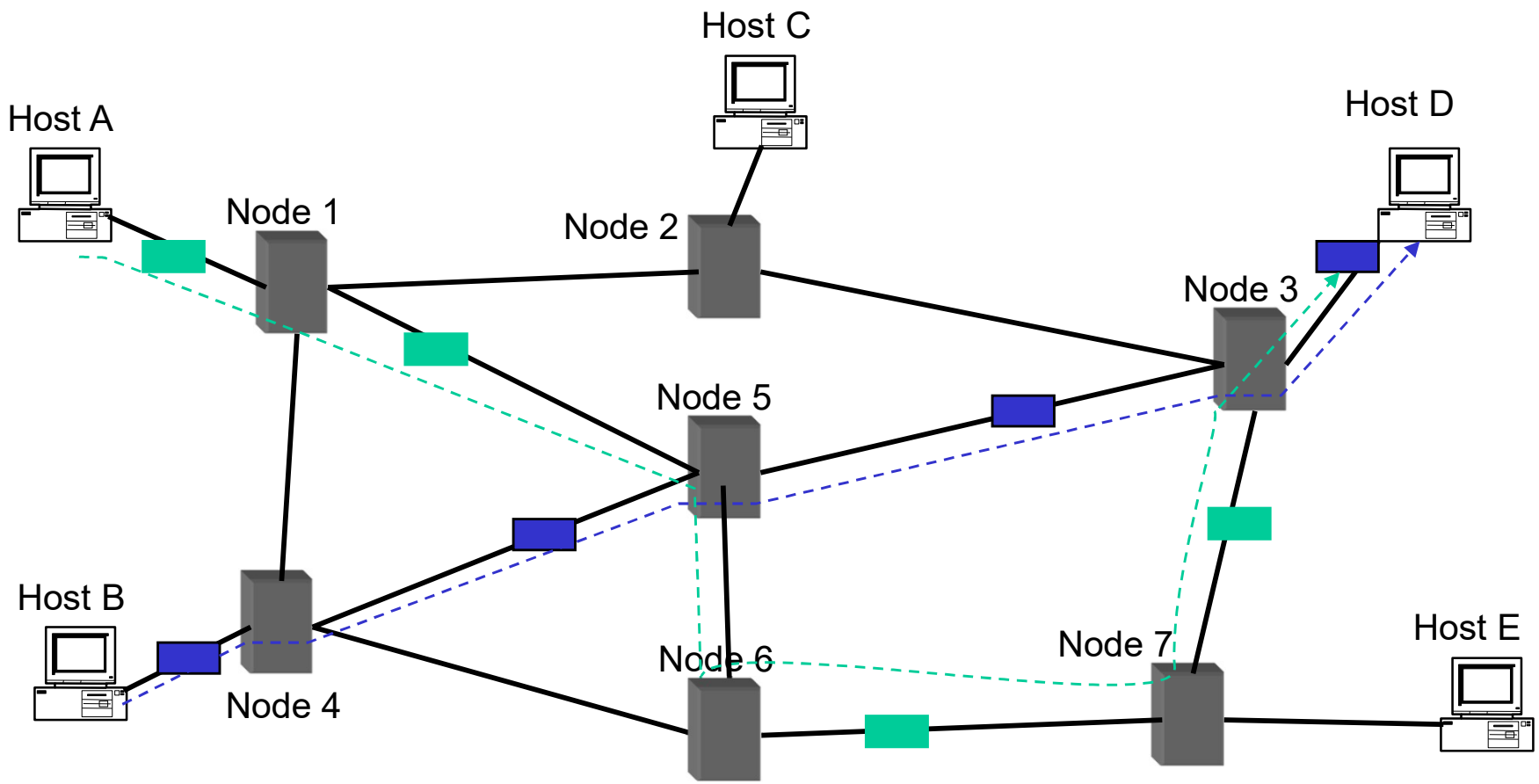
Mạng máy tính



Mạng máy tính

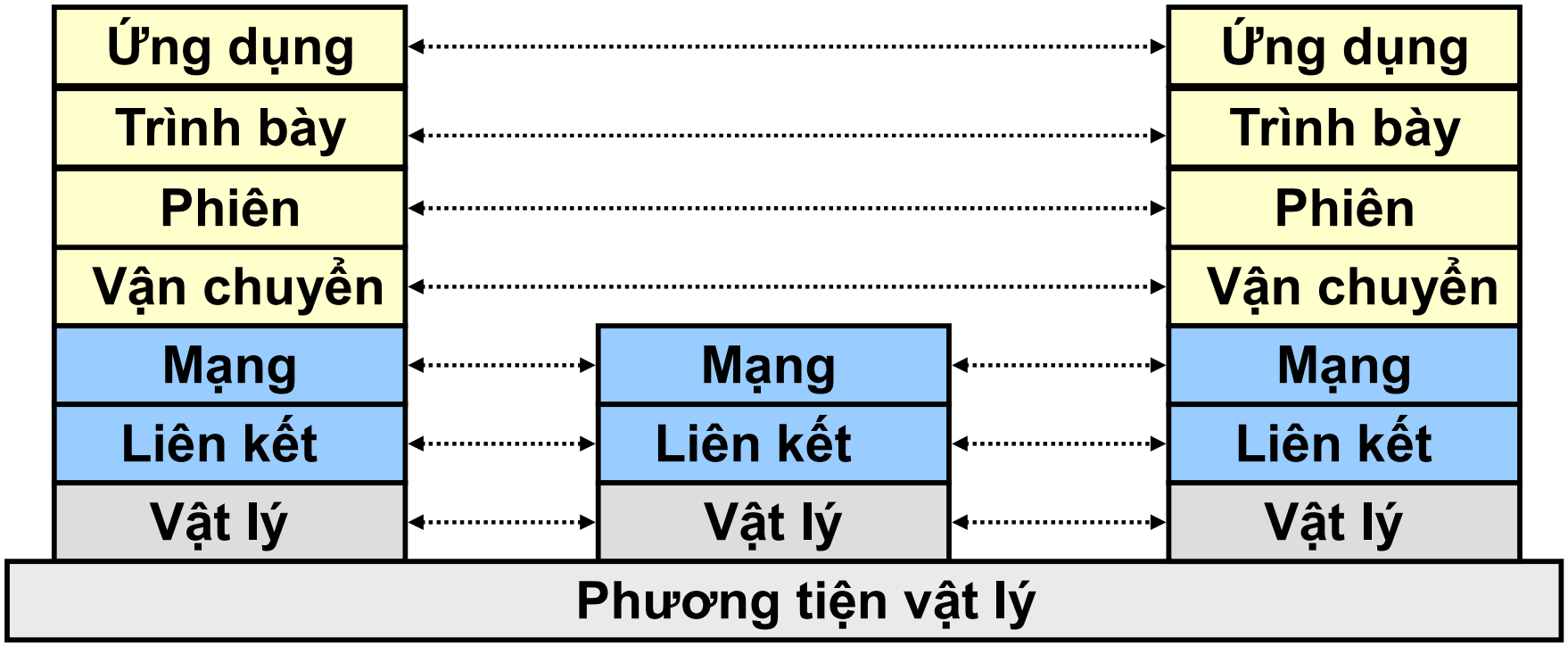


Mạng máy tính



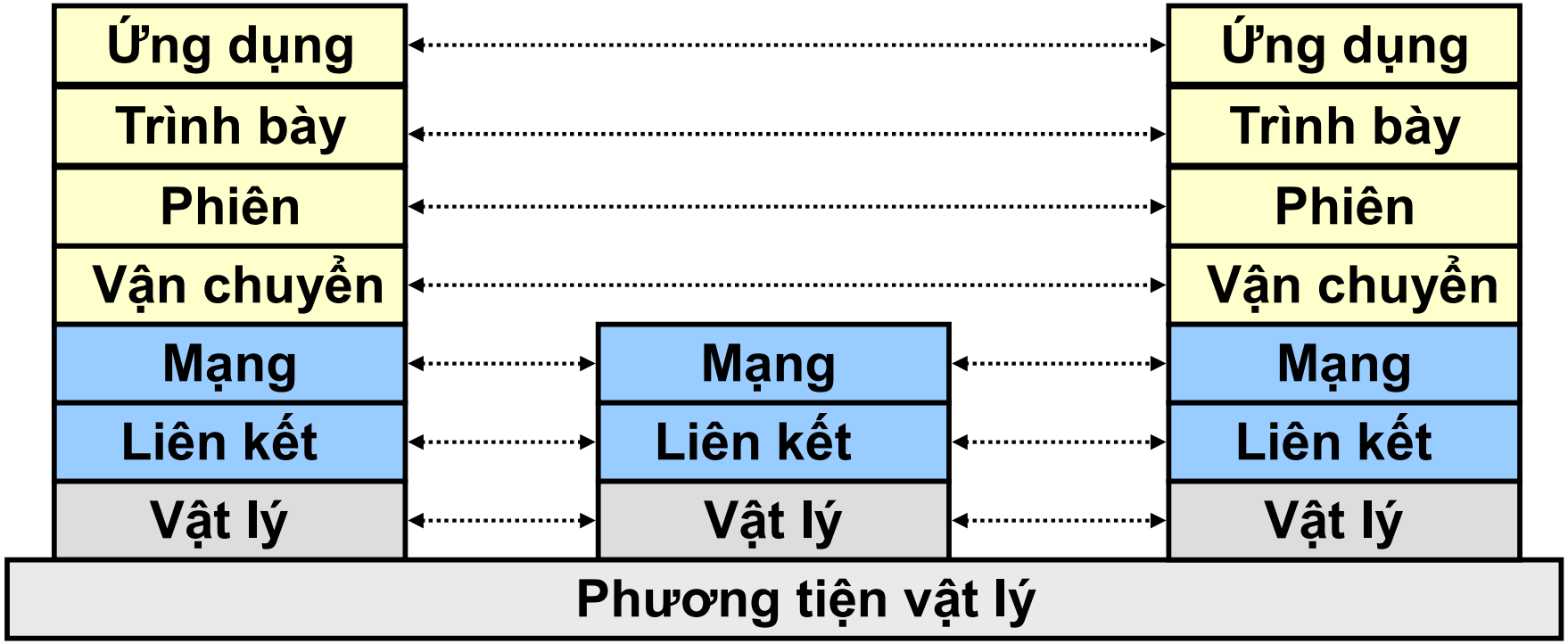
Mô hình OSI

- OSI
  - Cung cấp dịch vụ gì?
  - Giao diện - làm thế nào để sử dụng dịch vụ
  - Giao thức – dịch vụ được cài đặt như thế nào?



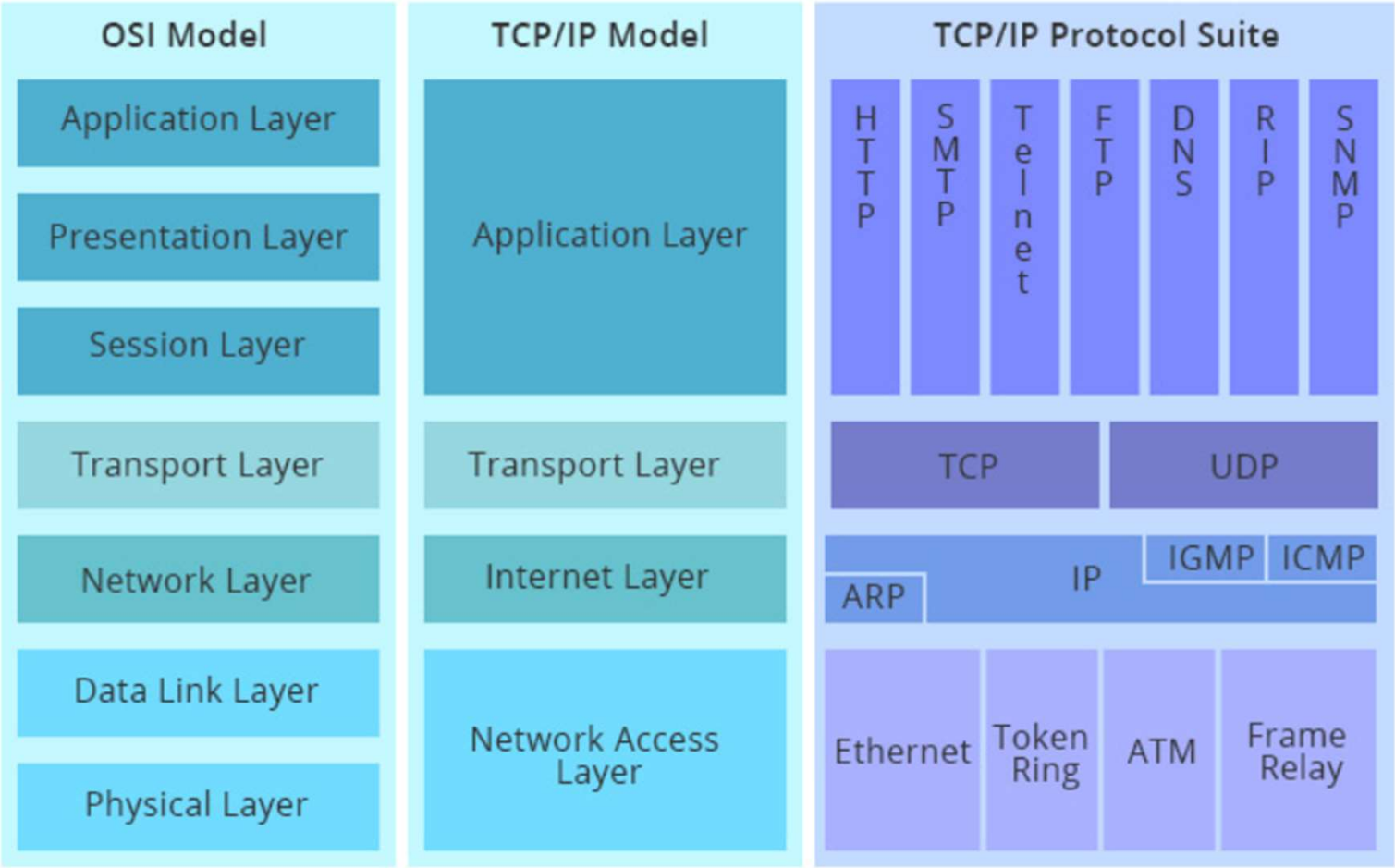
Mô hình OSI

- OSI



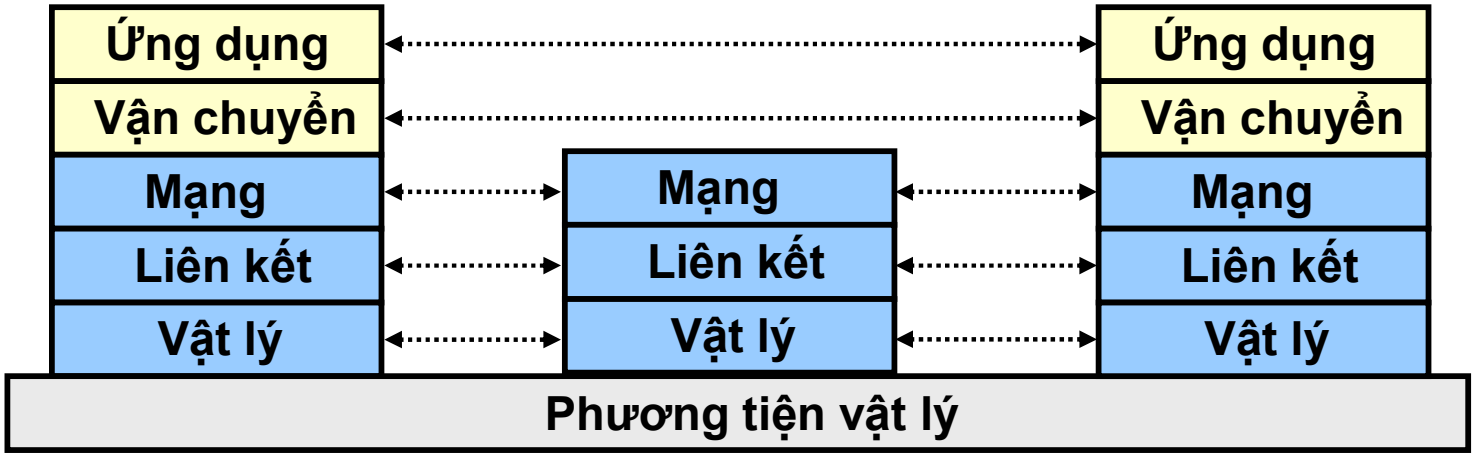
Mô hình OSI

- OSI vs TCP/IP



Mô hình Intenet(TCP/IP)

- TCP/IP



The Internet Protocol Suite

Application Layer

BGP · DHCP · DNS · **FTP** · GTP · HTTP · IMAP · IRC · Megaco · MGCP · NNTP · NTP · POP · RIP · RPC · RTP · RTSP · SDP · SIP · SMTP · SNMP · SOAP · SSH · Telnet · TLS/SSL · XMPP · (more)

Transport Layer

TCP · UDP · DCCP · SCTP · RSVP · ECN · (more)

Internet Layer

IP (IPv4, IPv6) · ICMP · ICMPv6 · IGMP · IPsec · (more)

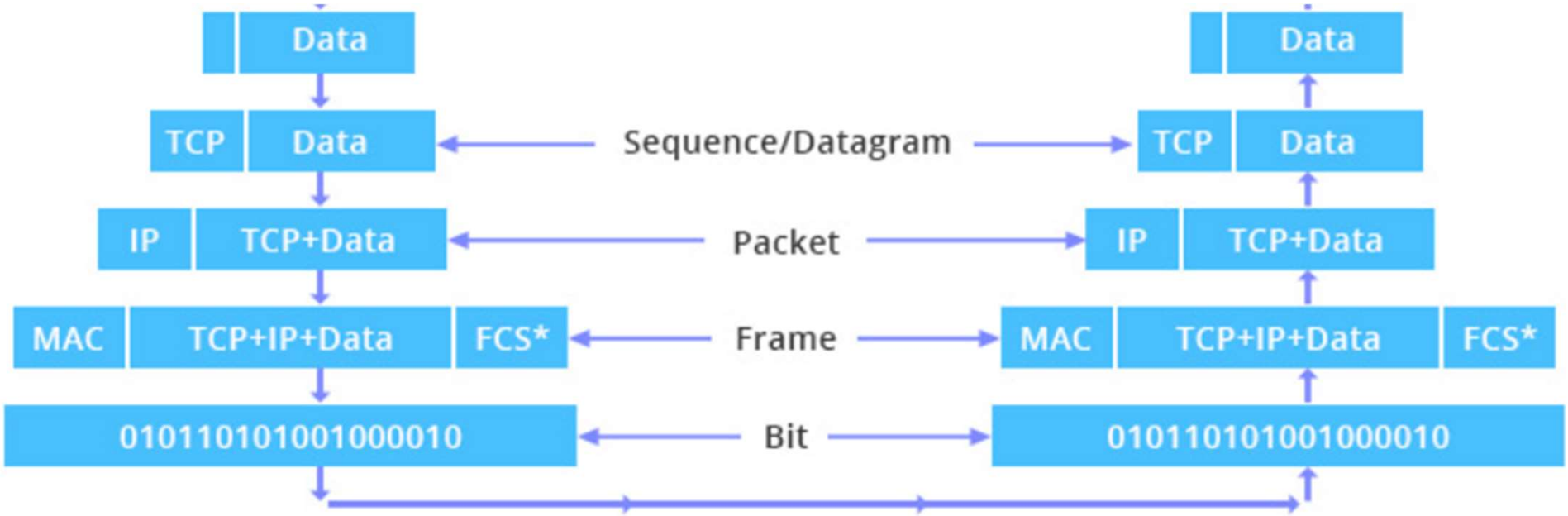
Link Layer

ARP · RARP · NDP · OSPF · Tunnels (L2TP) · PPP · Media Access Control (Ethernet, MPLS, DSL, ISDN, FDDI) · (more)

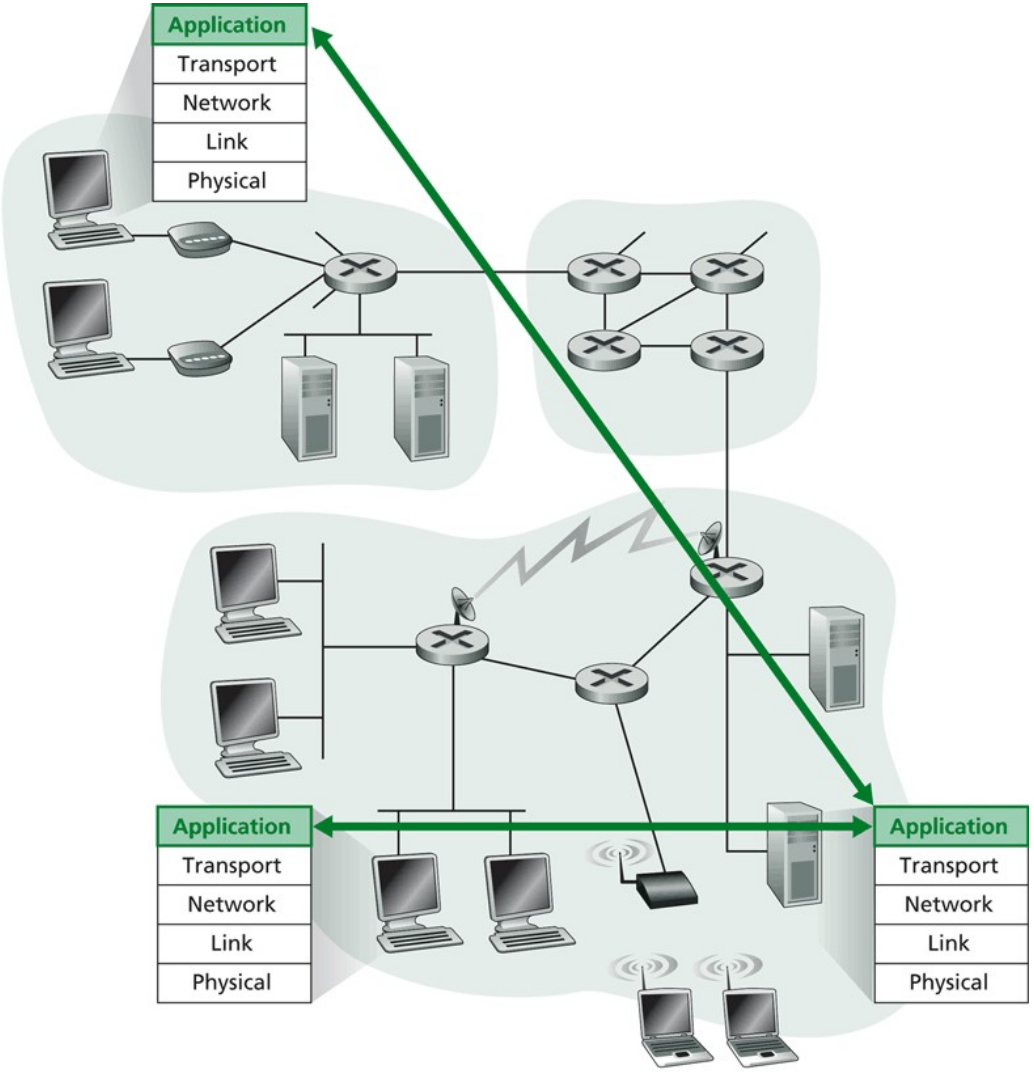
This box: view · talk · edit

Mô hình Intenet(TCP/IP)

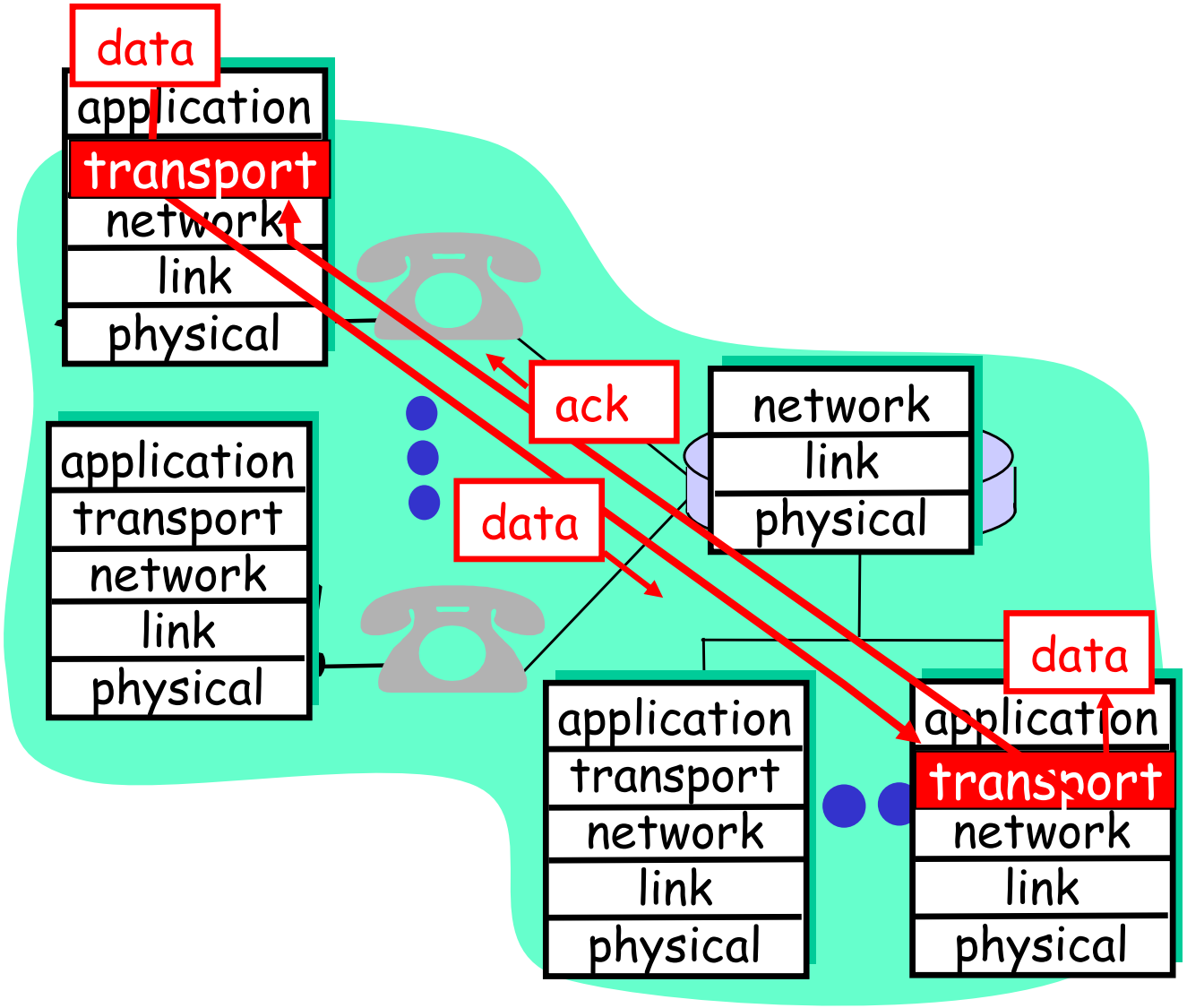
- TCP/IP



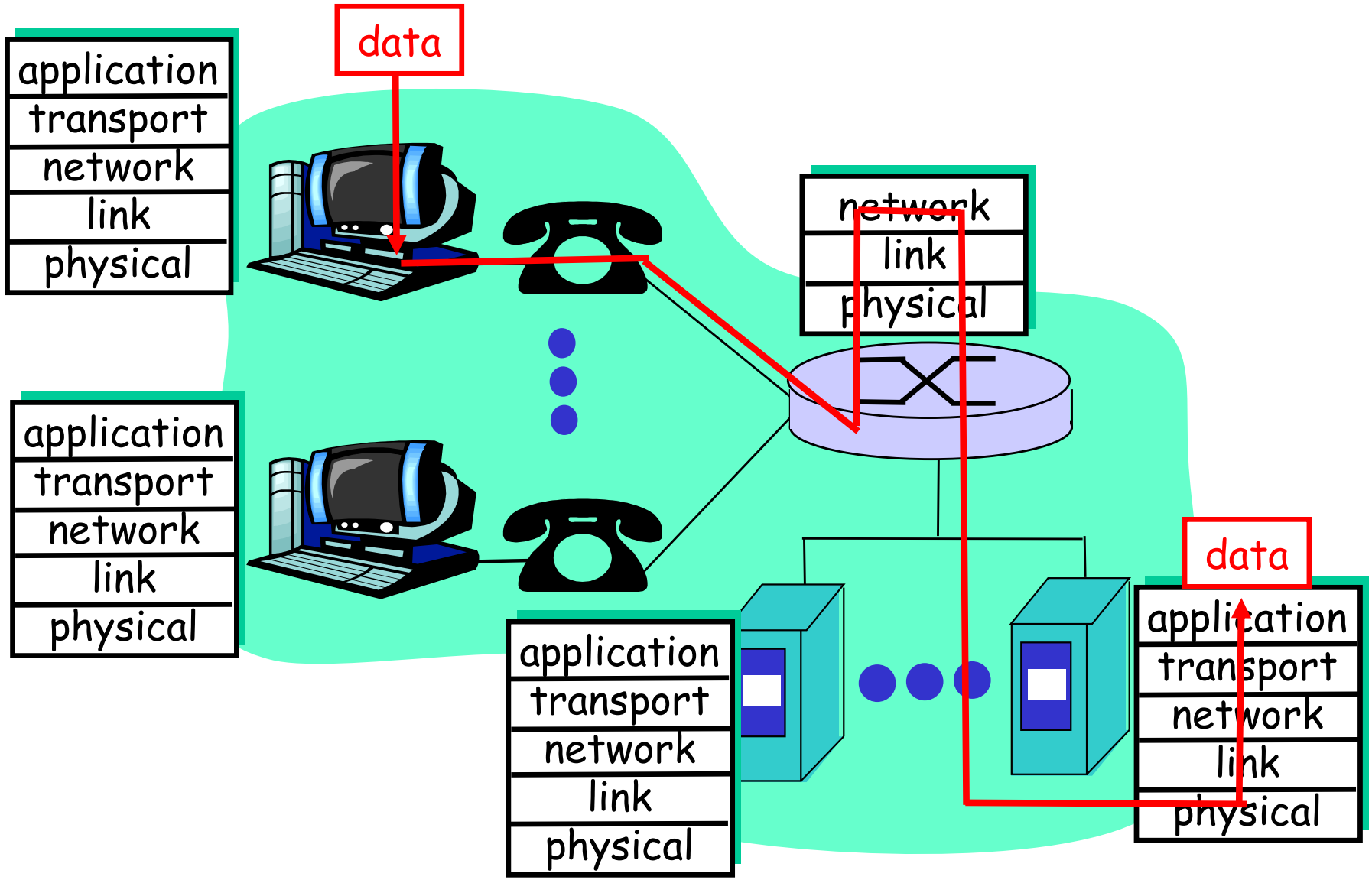
Ví dụ


Application
Transport
Network
Link
Physical
Application
Transport
Network
Link
Physical
Application
Transport
Network
Link
Physical

Ví dụ: biểu diễn giao tiếp logic



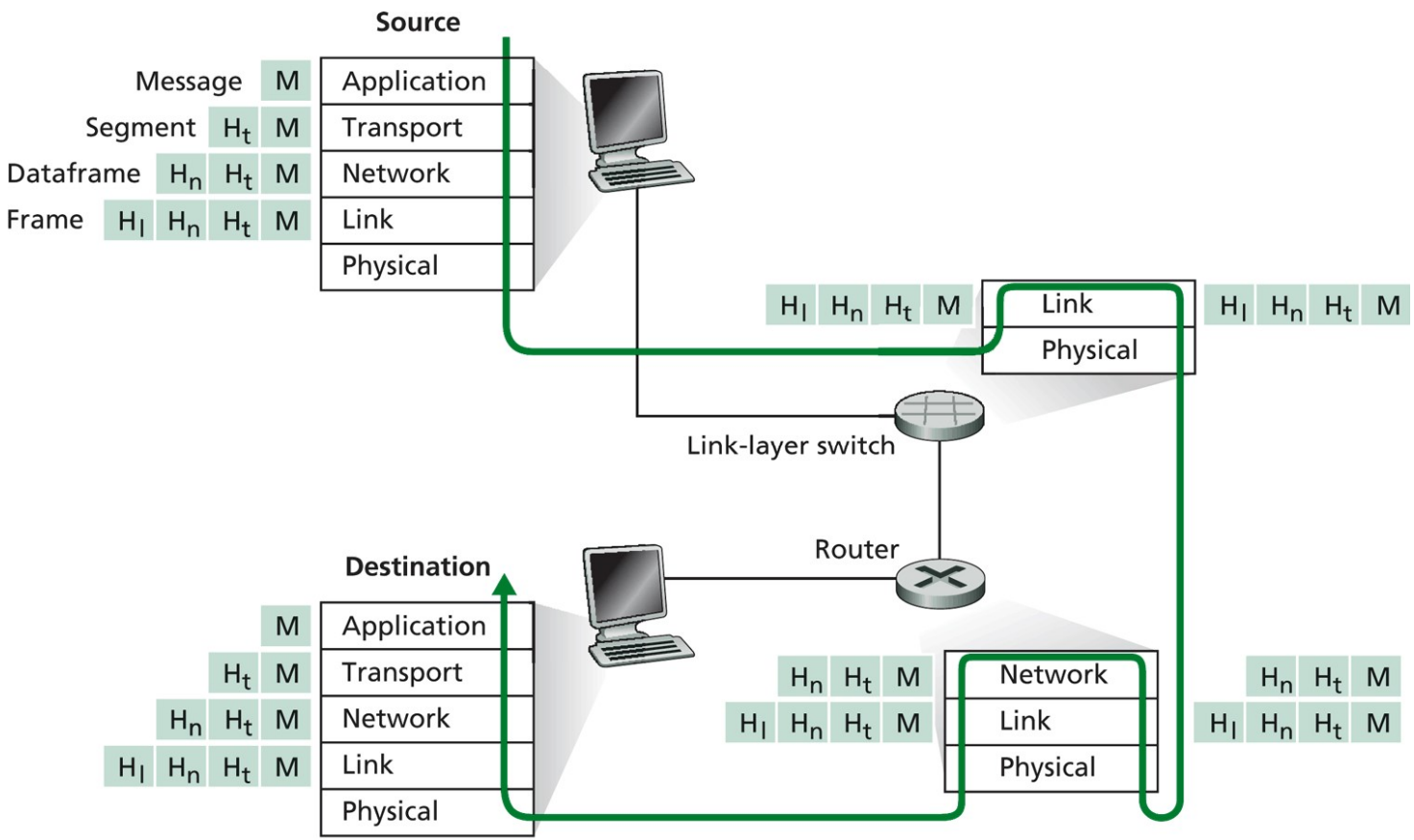
Ví dụ: biểu diễn giao tiếp vật lý



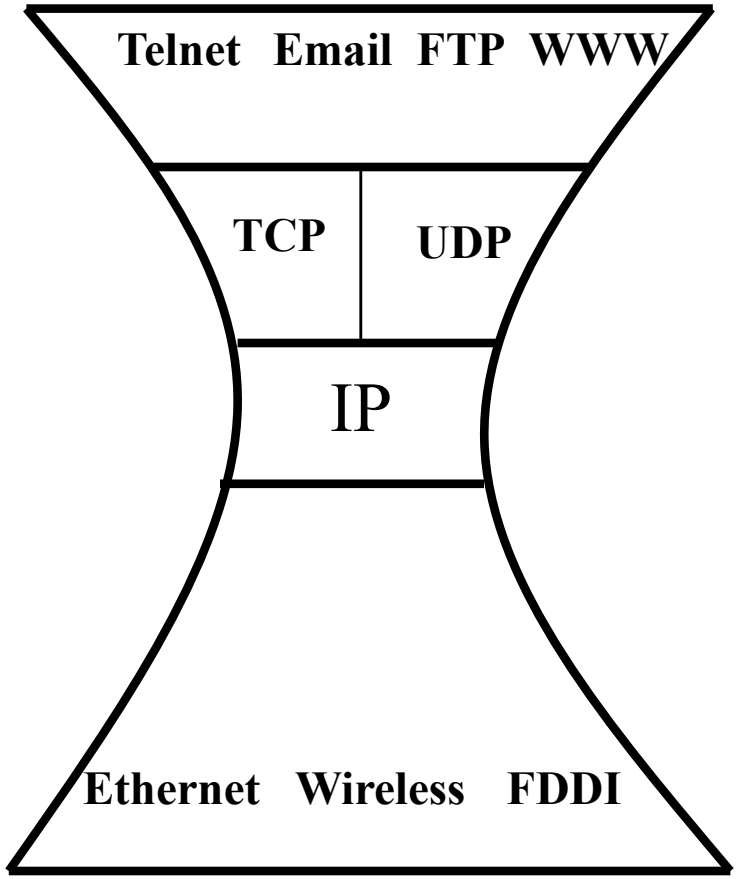
Ví dụ: dữ liệu và giao thức

Mỗi tầng lấy dữ liệu từ tầng trên

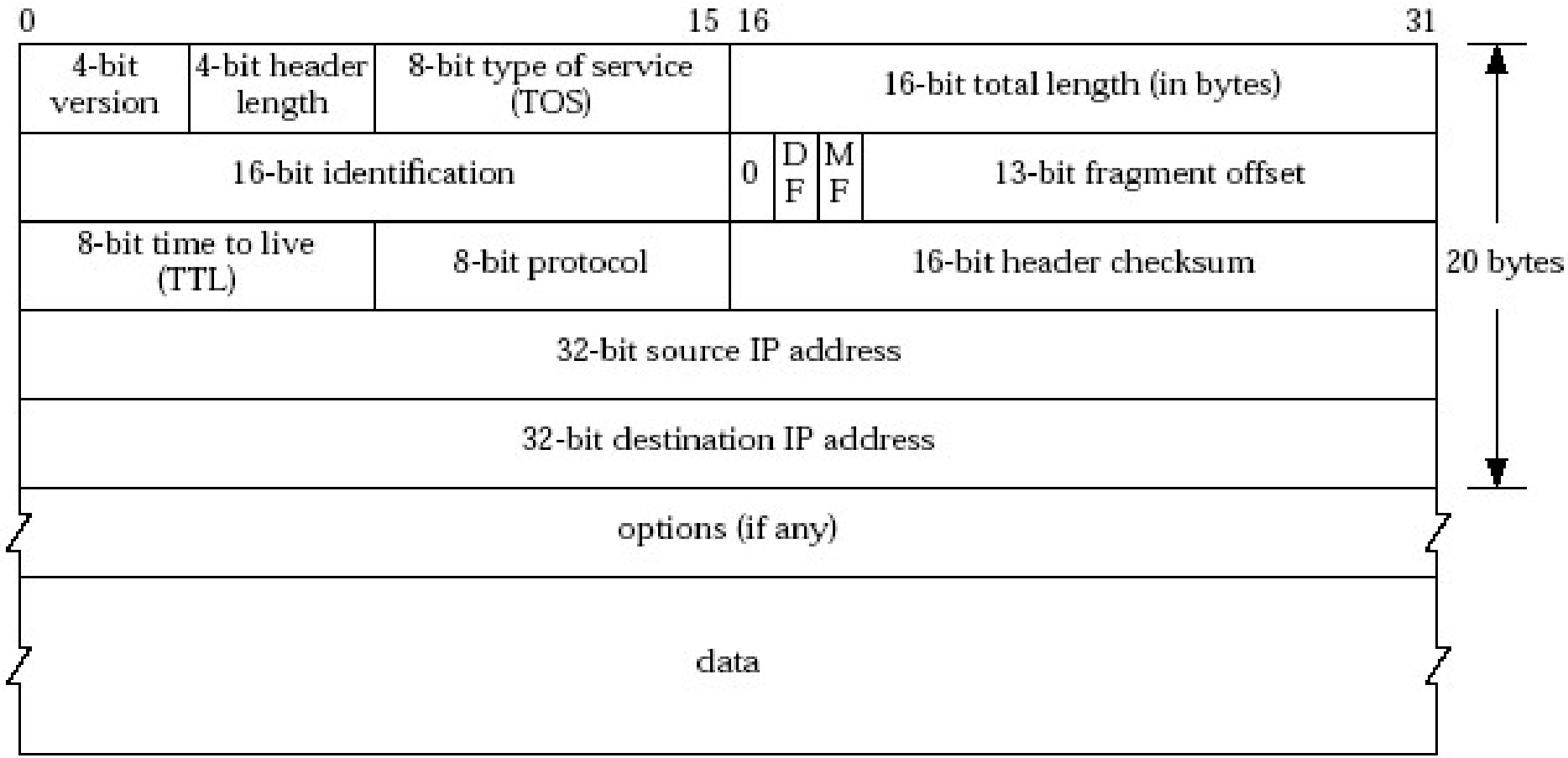
- Thêm thông tin header cho mỗi đơn vị dữ liệu
- Gởi dữ liệu mới tới tầng dưới



Kiến trúc đồng hồ cát (TCP/IP)



IPv4

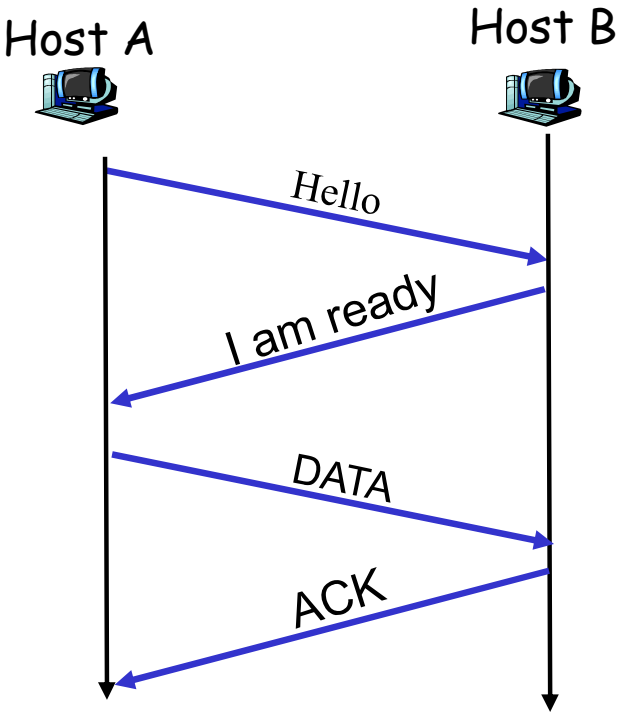

0
15
16
31
4-bit version
4-bit header length
8-bit type of service (TOS)
16-bit total length (in bytes)
16-bit identification
0
D F
M F
13-bit fragment offset
8-bit time to live (TTL)
8-bit protocol
16-bit header checksum
20 bytes
32-bit source IP address
32-bit destination IP address
options (if any)
data

Địa chỉ IPv4

Lớp	Cấu trúc địa chỉ IP	Format	Số bit mạng/số bit host	Tổng số mạng/lớp	Tổng số host/mạng	Vùng địa chỉ IP
A	0| netid|hostid	N.H.H.H	7/24	2^7-2=126	2^{24}-2=17.777.214	1.0.0.1-126.0.0.0
B	1|0| netid|hostid	N.N.H.H	14/16	2^{14}-2=16382	2^{16}-2=65.643	128.1.0.0-191.254.0.0
C	1|1|0| netid|hostid	N.N.N.H	22/8	2^{22}-2=4194302	2^8-2=245	192.0.1.0-223.255.254.0
D	1|1|1|0| địa chỉ Multicast	-	-	-	-	224.0.0.0-239.255.255.255
E	1|1|1|1	-	-	-	-	240.0.0.0-254.255.255.255
Loopback	-	-	-	-	-	127.x.x.x

TCP

Source port			Destination port
Sequence Number			
Acknowledge Number			
Offset	Reserved	Flags	Window
Checksum			Urgent pointer
Options			Padding
Start of Data			

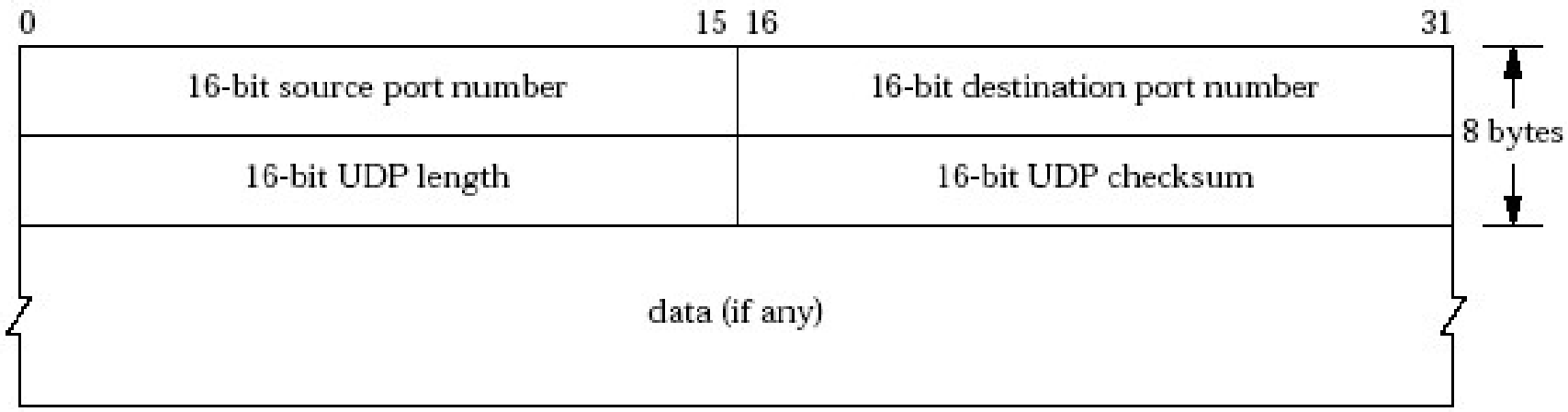


TCP

Trường	Mô tả
source port	Số hiệu cổng của nguồn
destination port	Số hiệu cổng đích
Sequence Number	Số thứ tự được tạo ra bởi nguồn
Acknowledge Number	Cho biết dữ liệu được nhận thành công.
Data offset	Các chi tiết về nơi dữ liệu gói tin bắt đầu
Reserved	Dự phòng
Flags	chỉ ra rằng gói tin cuối cùng hoặc gói khẩn cấp
Window	chỉ ra kích thước của vùng đệm nhận.
Checksum	xác định xem gói tin có bị hỏng không
Urgent Pointer	thông báo cho phía nhận biết có dữ liệu khẩn
Options	vùng dự phòng cho việc thiết lập trong tương lai
Padding	chỉ ra rằng dữ liệu kết thúc trong vòng 32 bit.

UDP

- Dịch vụ không kết nối
- Không cung cấp: thiết lập kết nối, tin cậy, điều khiển dòng, thời gian, …
- Tại sao sử dụng UDP?



UDP

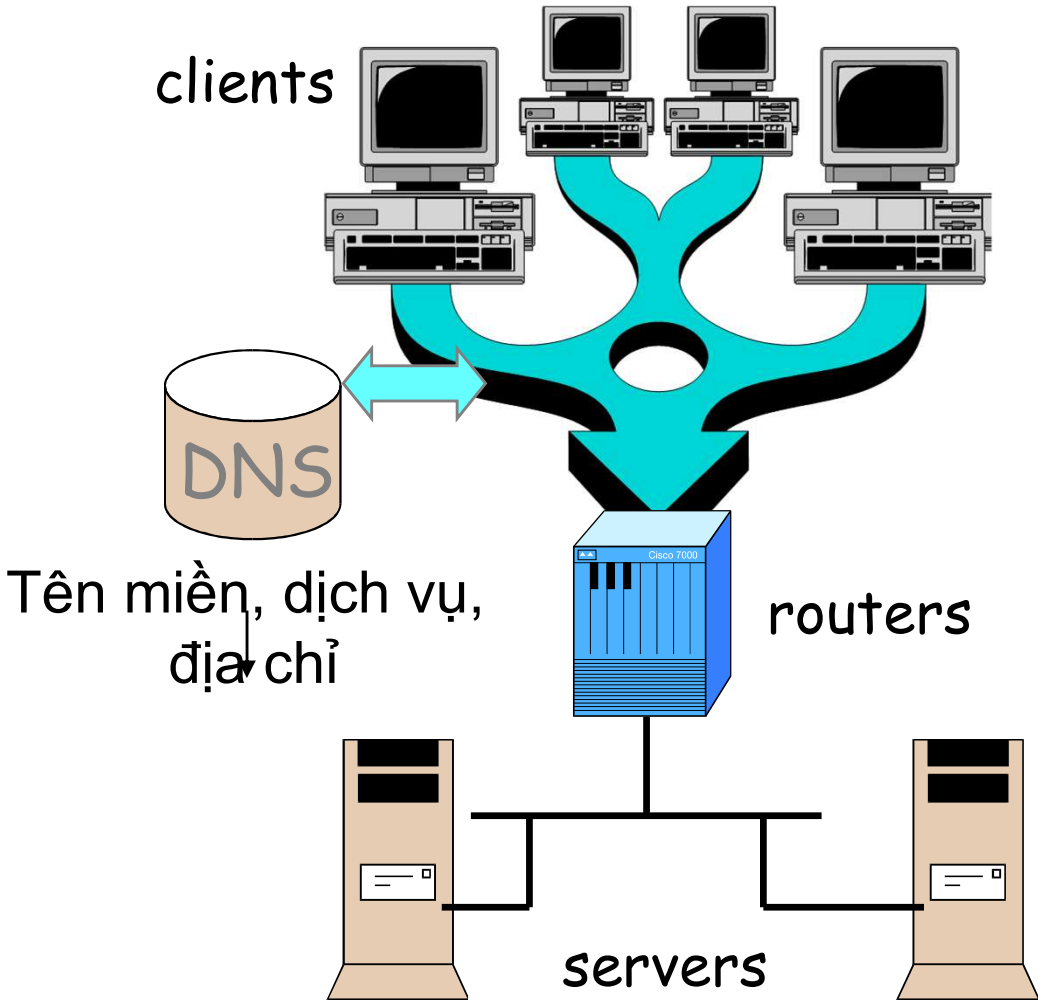
- Nhược điểm
  - Các thông điệp có thể được nhận theo bất kỳ thứ tự nào.
  - Không có gì đảm bảo là các gói tin sẽ đến đích
- Ưu điểm:
  - UDP một giao thức có tốc độ truyền tin nhanh
  - Truyền tin unicast, broadcast và multicast.
- Kiểu truyền
  - Một thông điệp unicast được gửi từ nút này tới nút khác.
  - Truyền tin broadcast nghĩa là một thông điệp có thể được gửi tới tất cả các nút trong một mạng.
  - Multicast cho phép các thông điệp được truyền tới một nhóm các nút được lựa chọn.

Số hiệu cổng TCP/UDP

- Có 3 loại (netstat –a)
  - số hiệu cổng hệ thống (0-1023). Các giao thức nổi tiếng có các số hiệu cổng nằm trong khoảng này.
  - Các số hiệu cổng người dùng (1024-49151). Các ứng dụng server của bạn sẽ nhận một trong các số này làm cổng, hoặc bạn có thể đăng ký số hiệu cổng với IANA .
  - Các số hiệu cổng riêng và động (49152-65535)

DNS: Domain Name System

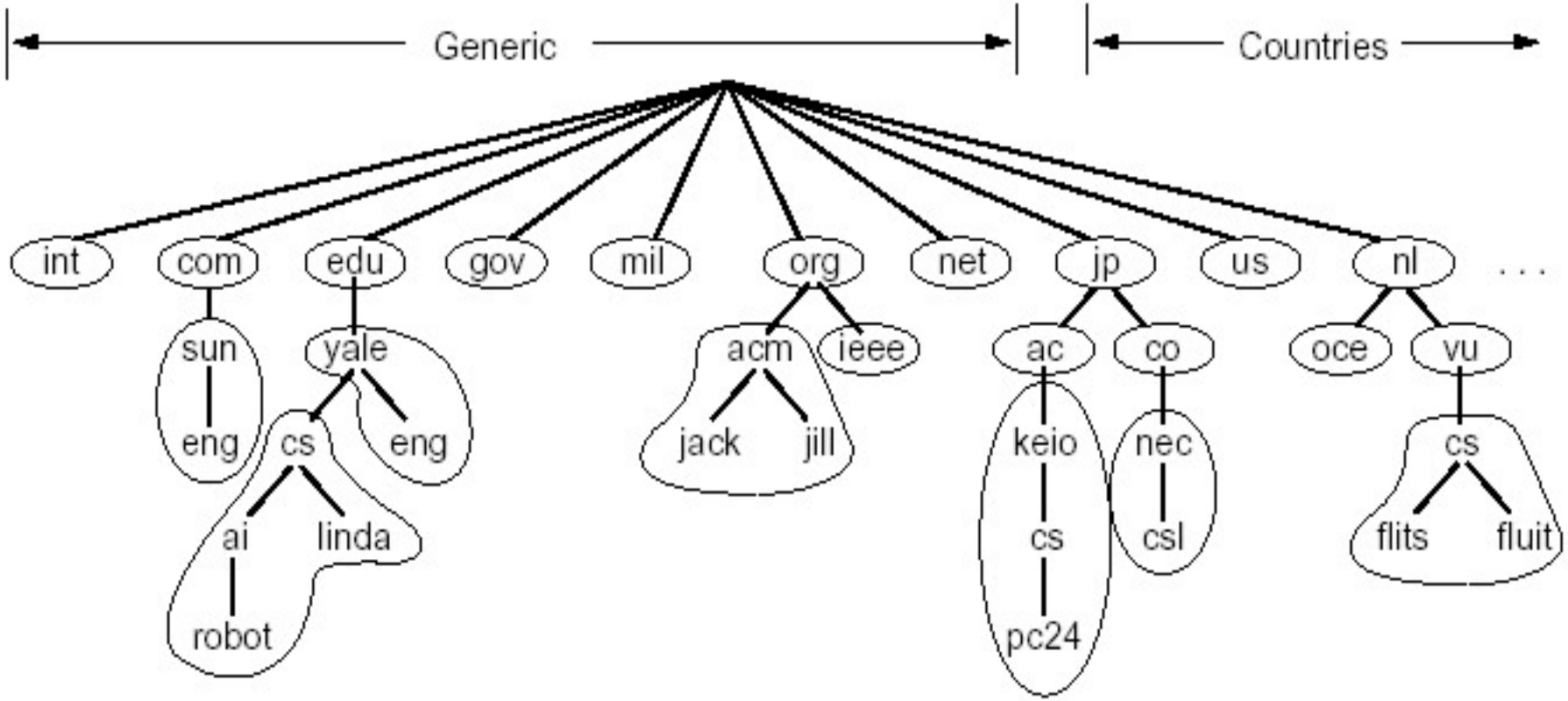
- Chức năng
  - Ánh xạ (tên miền, dịch vụ) sang giá trị, ví dụ,,
    - (www.cs.yale.edu, Addr) -> 128.36.229.30
    - (cs.yale.edu, Email) -> netra.cs.yale.edu
    - (netra.cs.yale.edu, Addr) -> 128.36.229.21



- Tại sao phải dùng tên thay cho địa chỉ IP?

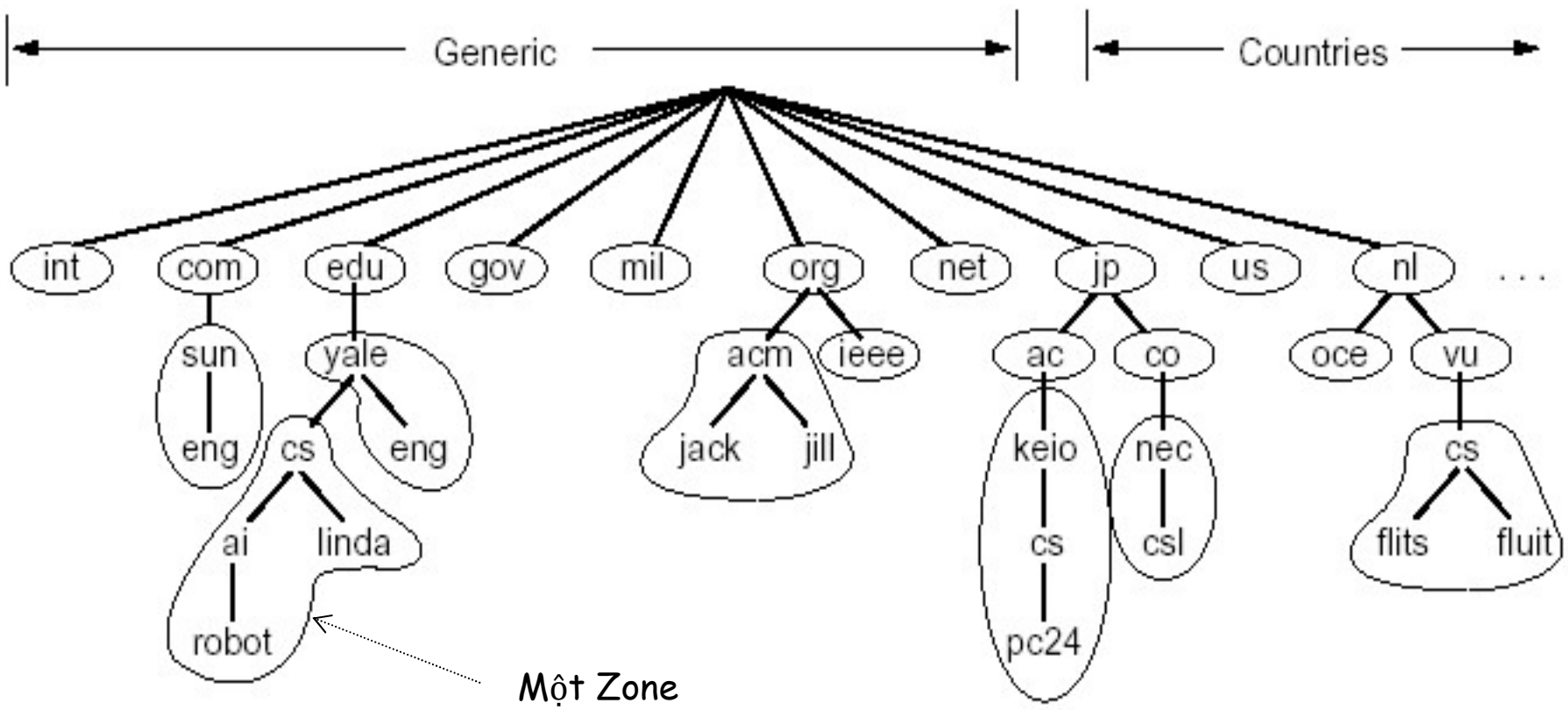
DNS: Domain Name System

- Lược đồ tên miền (IANA)
  - Nslookup (Domain/IP)
  - Ipconfig (DNS)



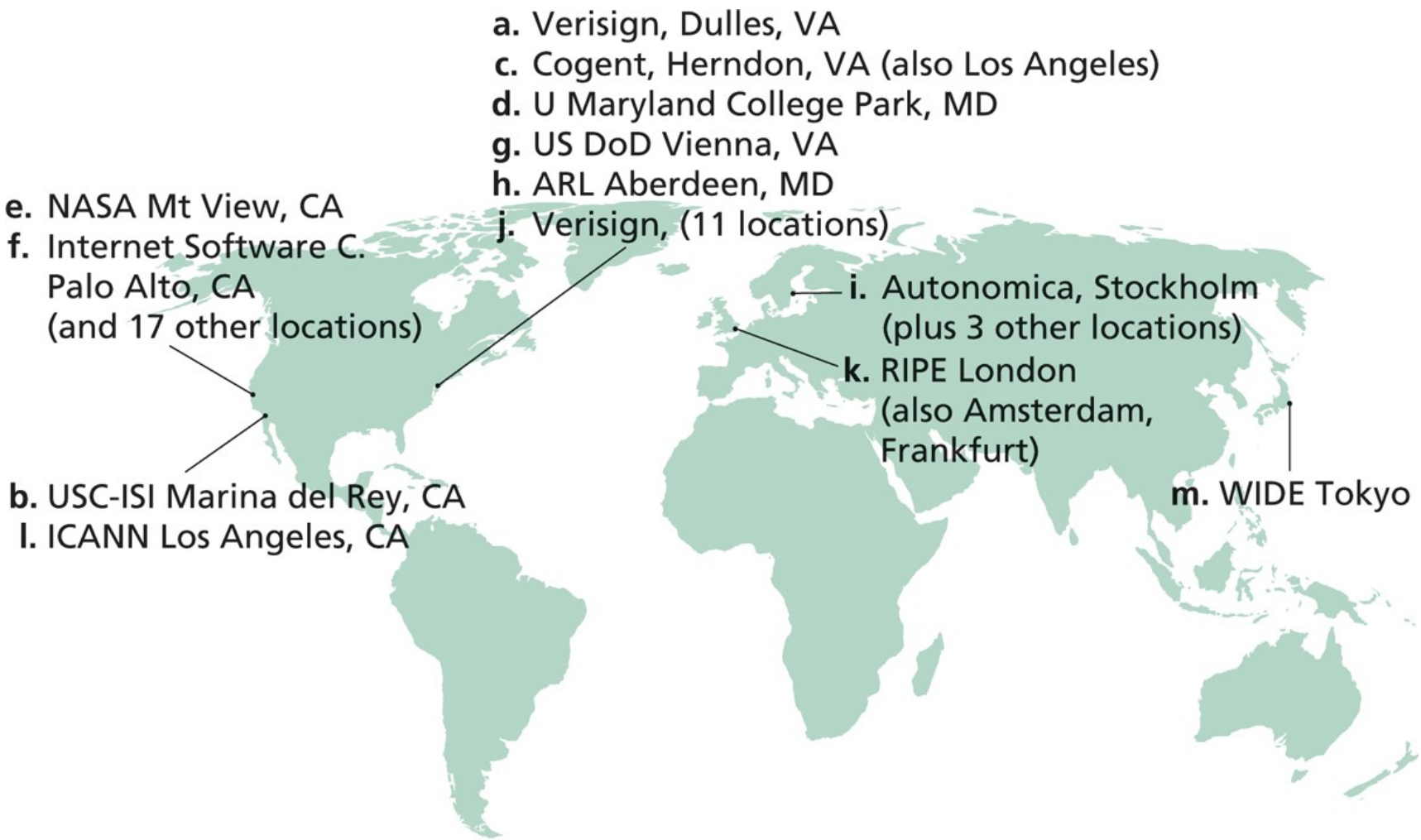
Quản lý tên miền

- A distributed database managed by authoritative name servers
  - Mỗi nhóm tên (Zone) có một server chứng thực tên
  - Mỗi Zone được ủy quyền chứng thực một tập tên miền.



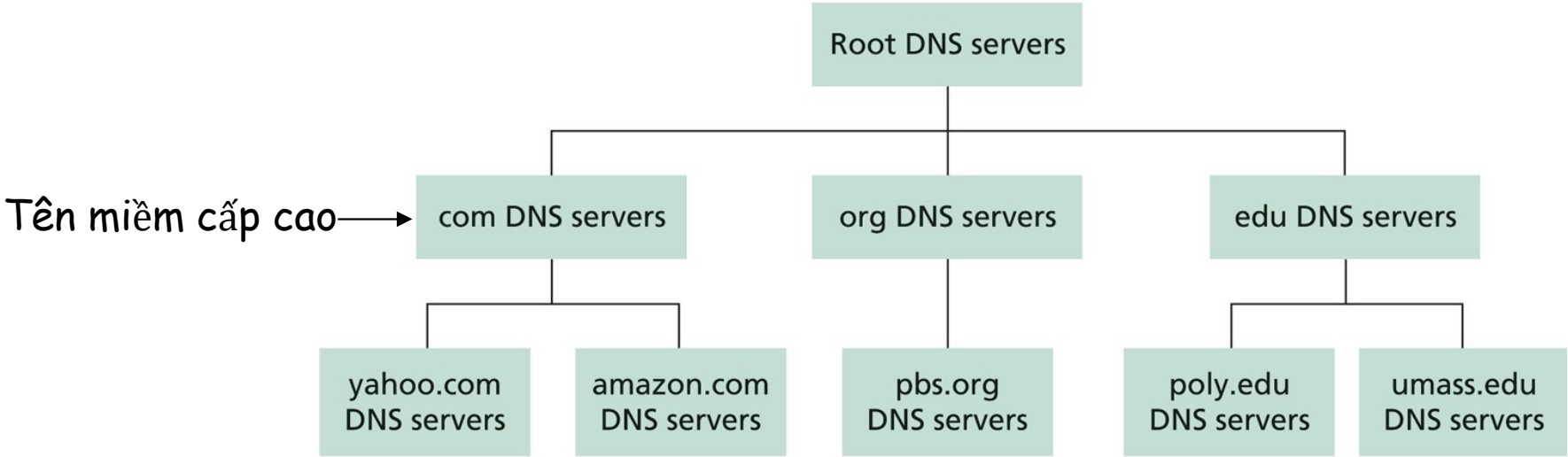
Nhóm Zone gốc và Server

- 13 servers quản lý zone gốc trên toàn cầu

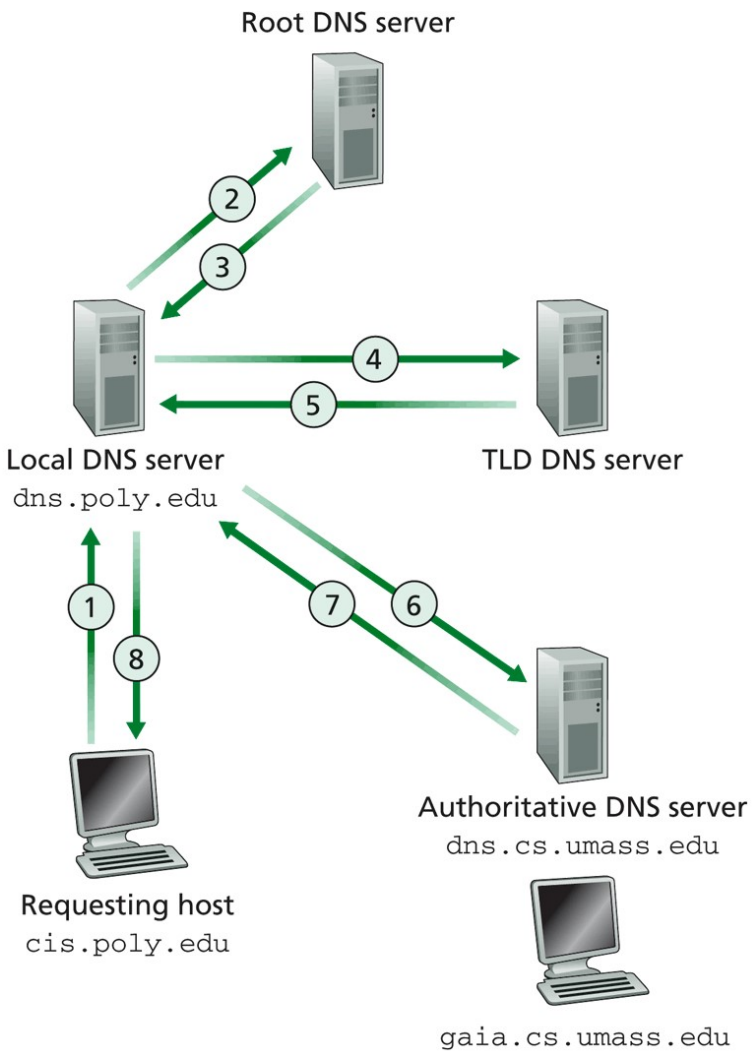


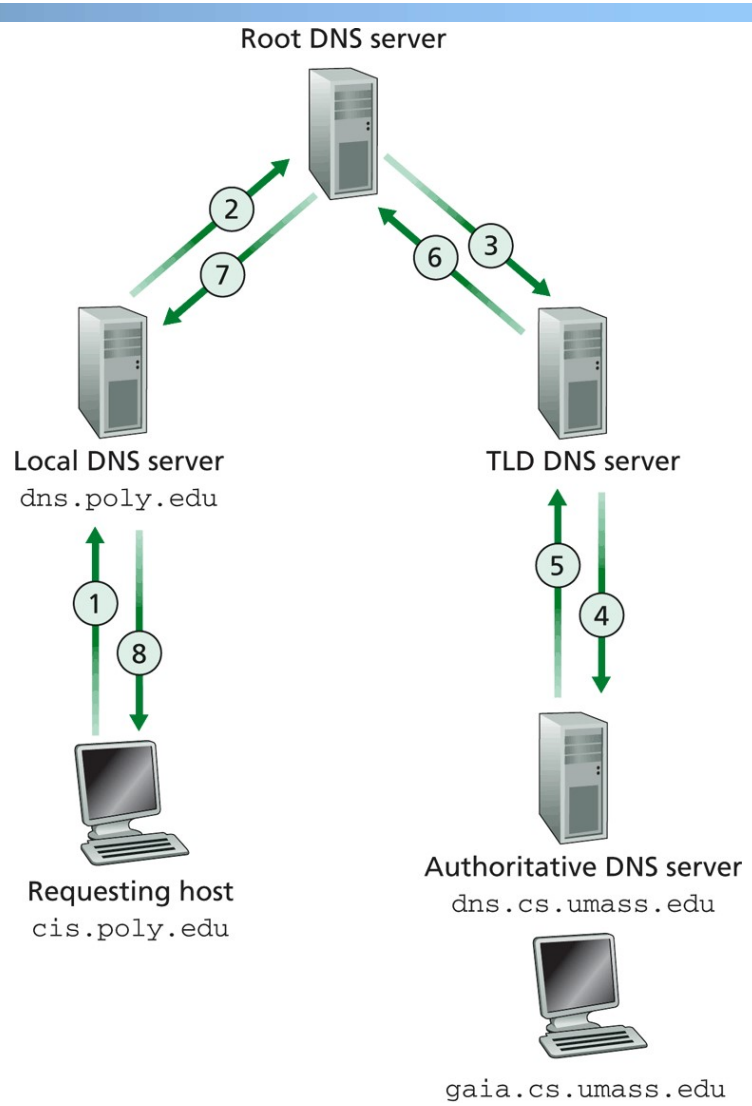
Liên kết giữa các sever DNS

- Mỗi server biết địa chỉ server gốc.
- Mỗi server gốc biết địa chỉ chính nó và địa chỉ node con trực tiếp



Ví dụ xác định tên miền





Bài tập

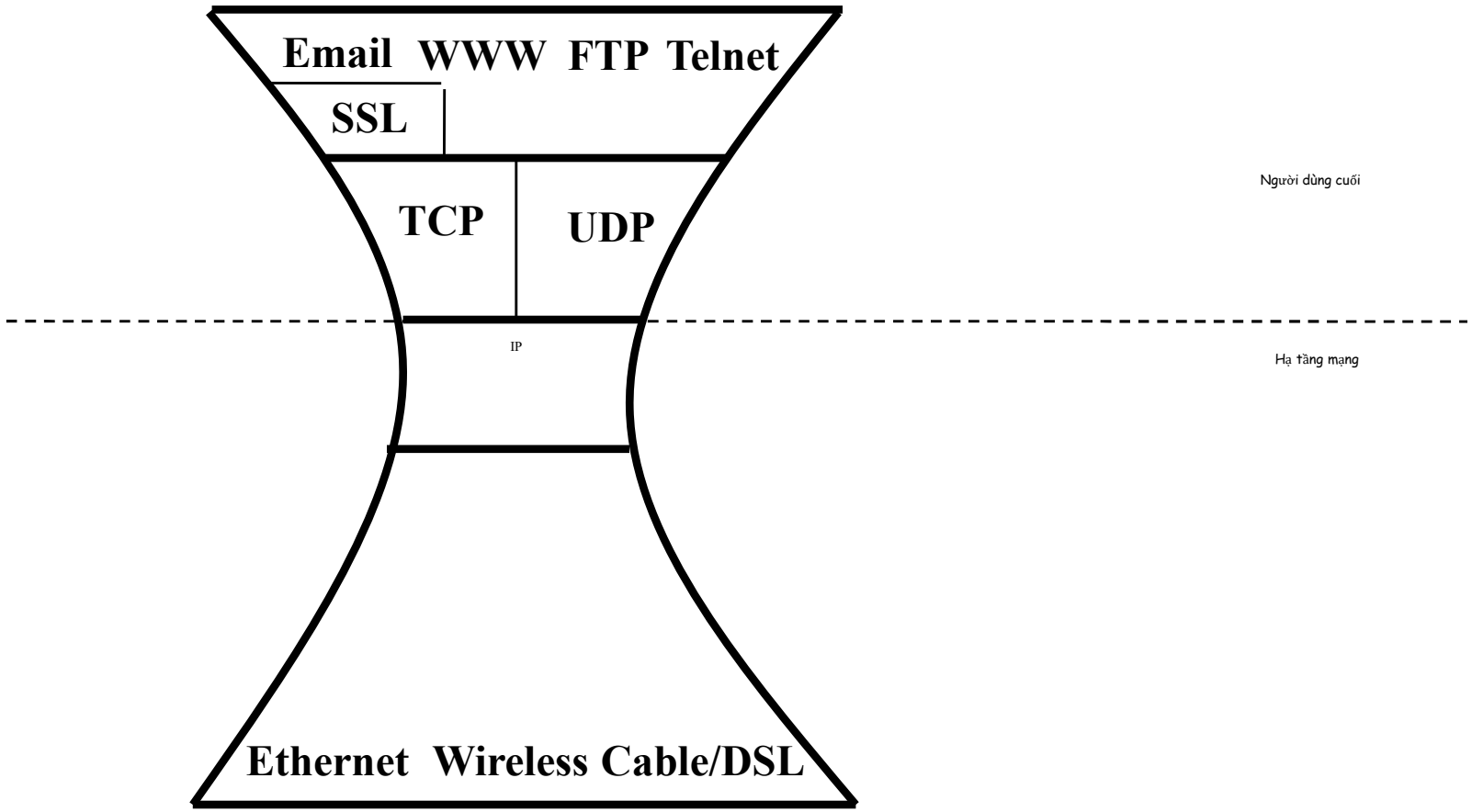
- Tìm hiểu các giao thức ở tầng ứng dụng HTTP, FTP, SMTP, POP3
  - Cho biết nội dung giao thức (khuôn dạng) giao tiếp qua lại giữa client/server
  - Lấy ví dụ minh họa
  - Vẽ hình minh họa
- Tìm tài liệu trên mạng: www.google.com

Các ứng dụng mạng

Nội dung

- Tổng quan tầng ứng dụng
- Ứng dụng tầng mạng
- Mô hình client-server
- Ví dụ

Kiến trúc mô hình Internet



Tầng ứng dụng

- Các khía cạnh về khái niệm và cài đặt của các giao thức tầng ứng dụng.
- Mô hình client-server
- Biết một số giao thức thông dụng của tầng mạng.
  - smtp/pop3
  - http
  - ftp
  - dns

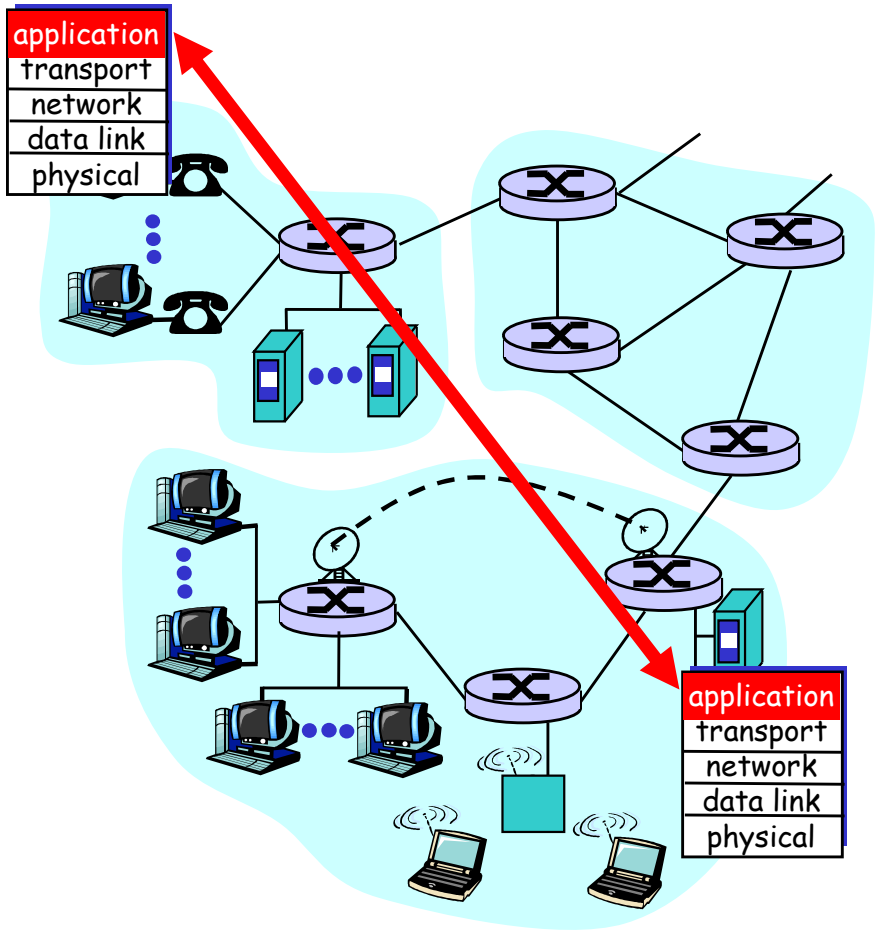
Ứng dụng mạng và giao thức tầng mạng

Các ứng dụng mạng

- Các chương trình chạy ở trên host cung cấp giao diện cho người dùng.
  - web: browser
  - audio/video: media player
- Sử dụng giao thức của tầng ứng dụng để giao tiếp.
- Ví dụ: email, Web

Các giao thức tầng ứng dụng

- Mẫu thông tin
- Định nghĩa cách thức giao tiếp giữa các ứng dụng.
- Thực hiện được dịch vụ với các yêu cầu trong giao thức bằng cách sử dụng các dịch vụ của tầng thấp hơn.



Sử dụng các dịch vụ tầng mạng, vận chuyển như thế nào?

Cung cấp một giao diện lập trình - API

- Định nghĩa giao diện giữa tầng ứng dụng và tầng mạng.
- Ví dụ: Socket API
  - Còn gọi là "Berkeley sockets".
  - Một socket bao gồm địa chỉ IP và số hiệu cổng
    - Ví dụ: email (SMTP) cổng 25, web (HTTP) cổng 80
  - Một ứng dụng sẽ gắn với một socket
    - %netstat –a
  - Hai chương trình giao tiếp với nhau bằng cách gởi dữ liệu tới socket, và đọc dữ liệu ra từ nó.
- Một số API khác như XTI (X/Open Transport Interface), (TLI) được phát triển bởi AT&T.

Chi tiết trình bày bài sau!

Ứng dụng và Giao thức

- Một ứng dụng cần chọn giao thức vận chuyển phù hợp

Ứng dụng	Giao thức tầng ứng dụng	Giao thức tầng vận chuyển bên dưới
		TCP/SSL
e-mail	smtp [RFC 821]	TCP
remote terminal access	telnet [RFC 854]	TCP/SSL
Web	http [RFC 2068]	TCP
file transfer	ftp [RFC 959]	UDP
Internet telephony	proprietary (ví dụ, Vocaltec)	TCP hoặc UDP
remote file server	NFS	UDP
streaming multimedia	proprietary	

Mô hình Client-Server

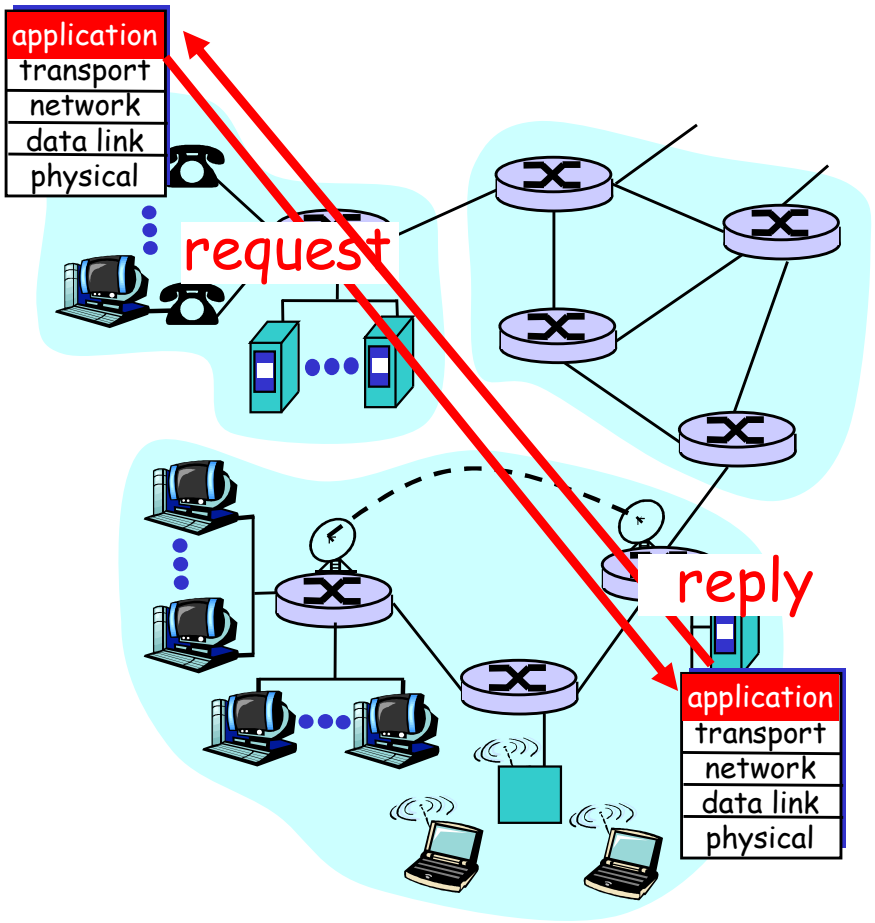
Có 2 loại ứng dụng mạng: *client* and *server*

Client (C):

- Thiết lập kết nối với server ("nói trước")
- Yêu cầu dịch vụ của server
- Ví dụ Web, client là trình duyệt web, e-mail, là mail reader (MS Outlook)

Server (S):

- Cung cấp các dịch vụ được yêu cầu cho client
- Ví dụ, Web server gởi trang Web được yêu cầu; mail server đáp trả e-mail

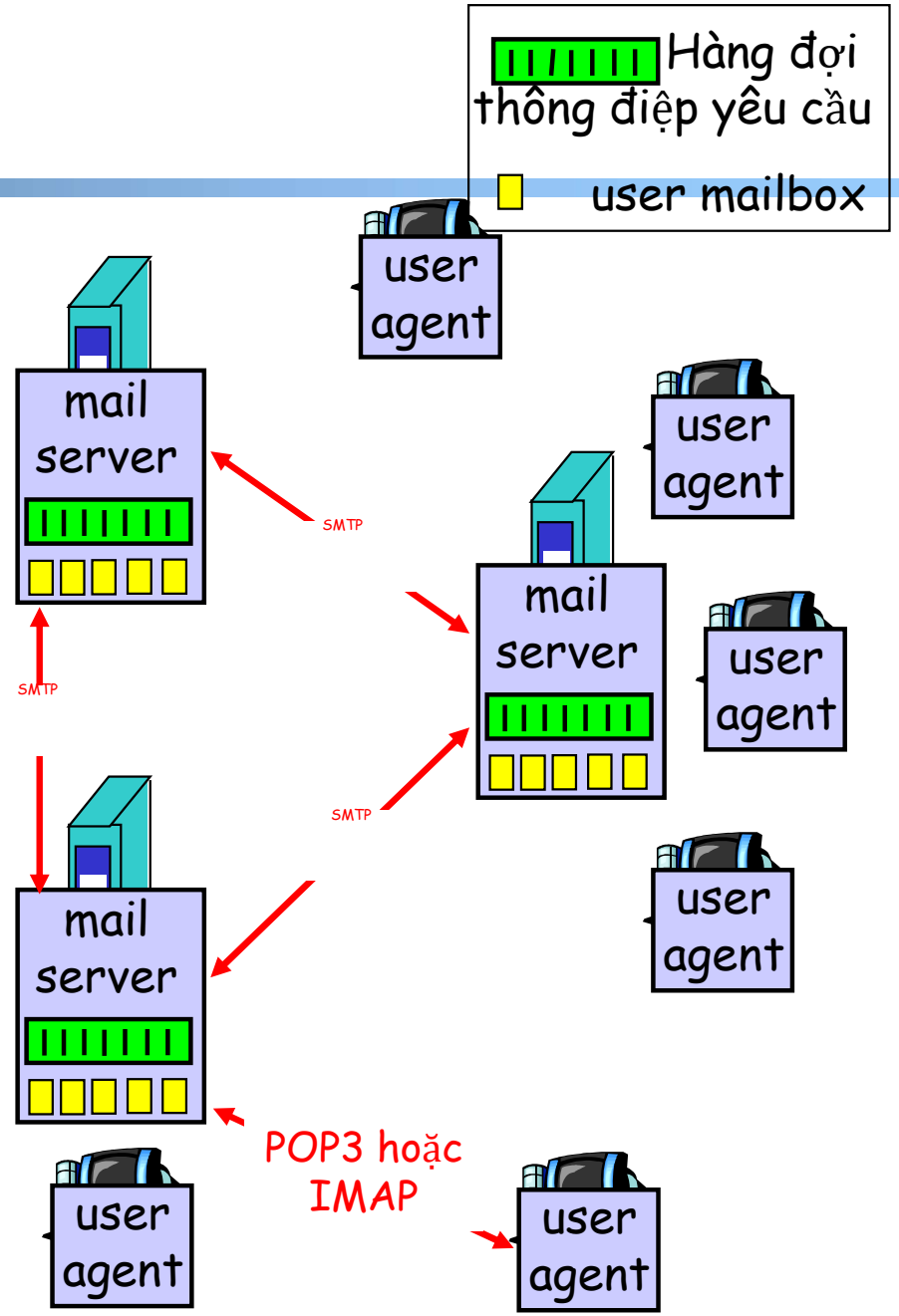


Câu hỏi đặt ra cho ứng dung C-S.
- Làm thế nào client định vị được Server?

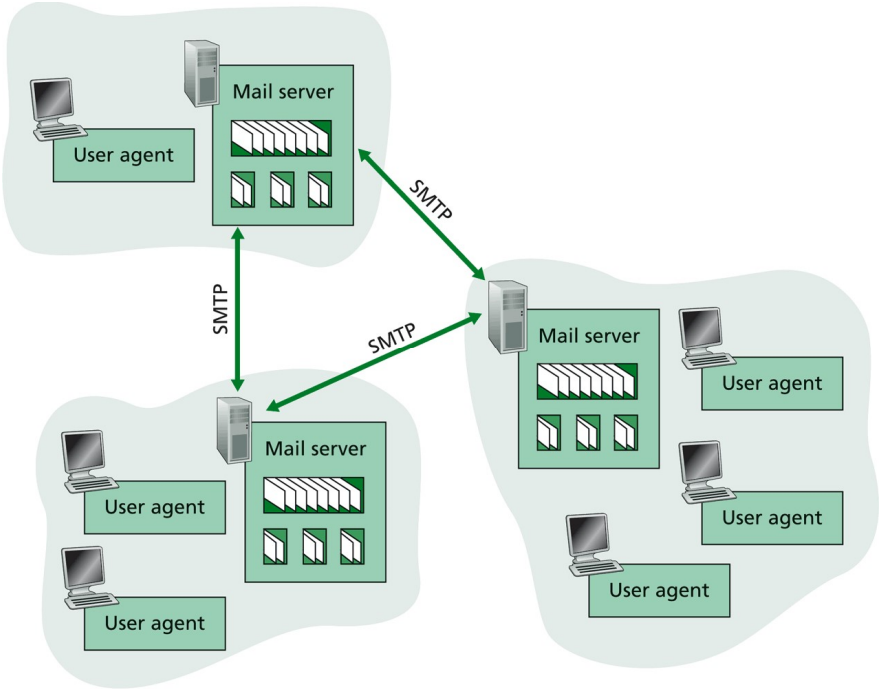
E-Mail

Gồm 3 thành phần:

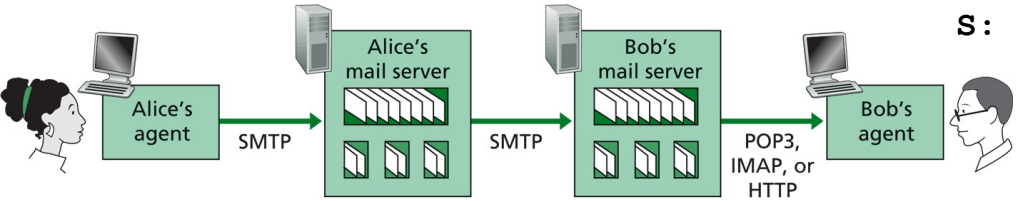
- User agents
- Mail servers
- Các giao thức
  - Gởi mail giữa các mail server
    - SMTP
  - Lấy mail giữa mail server and user agent
    - POP3: Post Office Protocol [RFC 1939]
    - IMAP: Internet Mail Access Protocol [RFC 1730]



Giao thức gởi mail - SMTP




```
S: 220 mr1.its.yale.edu
C: HELO cyndra.yale.edu
S: 250  Hello cyndra.cs.yale.edu, pleased to meet you
C: MAIL FROM: <spoof@cs.yale.edu>
S: 250 spoof@cs.yale.edu... Sender ok
C: RCPT TO: <yry@yale.edu>
S: 250 yry@yale.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Date: Wed, 23 Jan 2008 11:20:27 -0500 (EST)
C: From: "Y. R. Yang" <yry@cs.yale.edu>
C: To: "Y. R. Yang" <yry@cs.yale.edu>
C: Subject: This is subject
C:
C: This is the message body!
C: Please don't spoof!
C:
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 mr1.its.yale.edu closing connection
```




```
%telnet mr1.its.yale.edu 25
```

Bài tập về nhà :

- **Tìm hiểu** RFC 822 **?**
- **Tìm hiểu** RFC 2045, 2056**?**

Giao thức truy cập mail - POP3

Giai đoạn chứng thực

- Các lệnh client:
  - **user**
  - **pass**
- Đáp trả của server
  - **+OK**
  - **-ERR**

```
S: +OK POP3 server ready
C: user alice
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

Giai đoạn giao dịch, client:

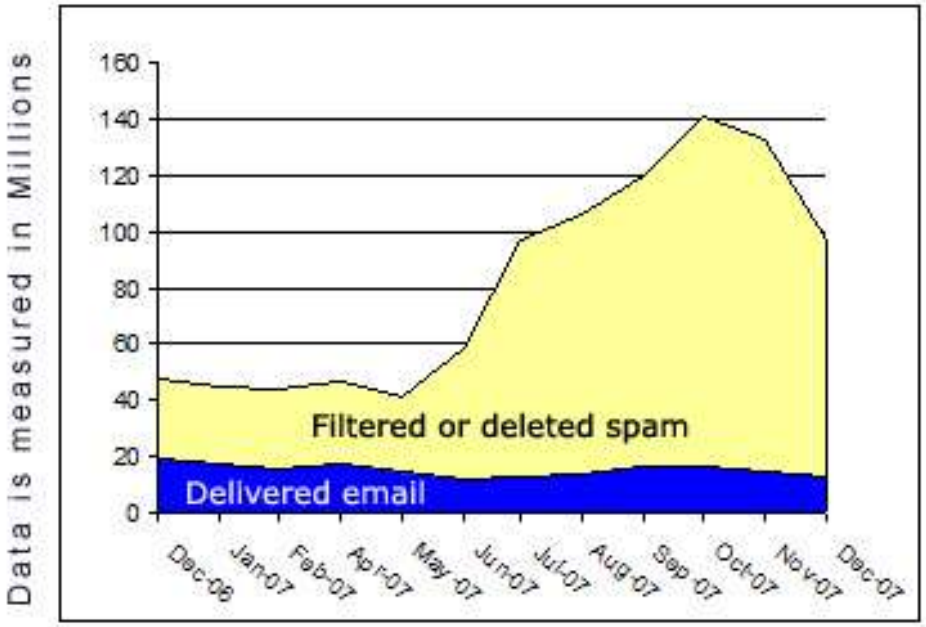
- **list:** xem ds thông điệp
- **retr:** lấy nội dung thông điệp
- **dele:** xóa thông điệp
- **quit:** đóng kết nối

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

telnet mail.hcmtdtu.edu.vn 110

Bài tập

- Tìm hiểu về Spam.
- Các cơ chế lọc spam và ứng dụng lọc spam hiện nay.



Web

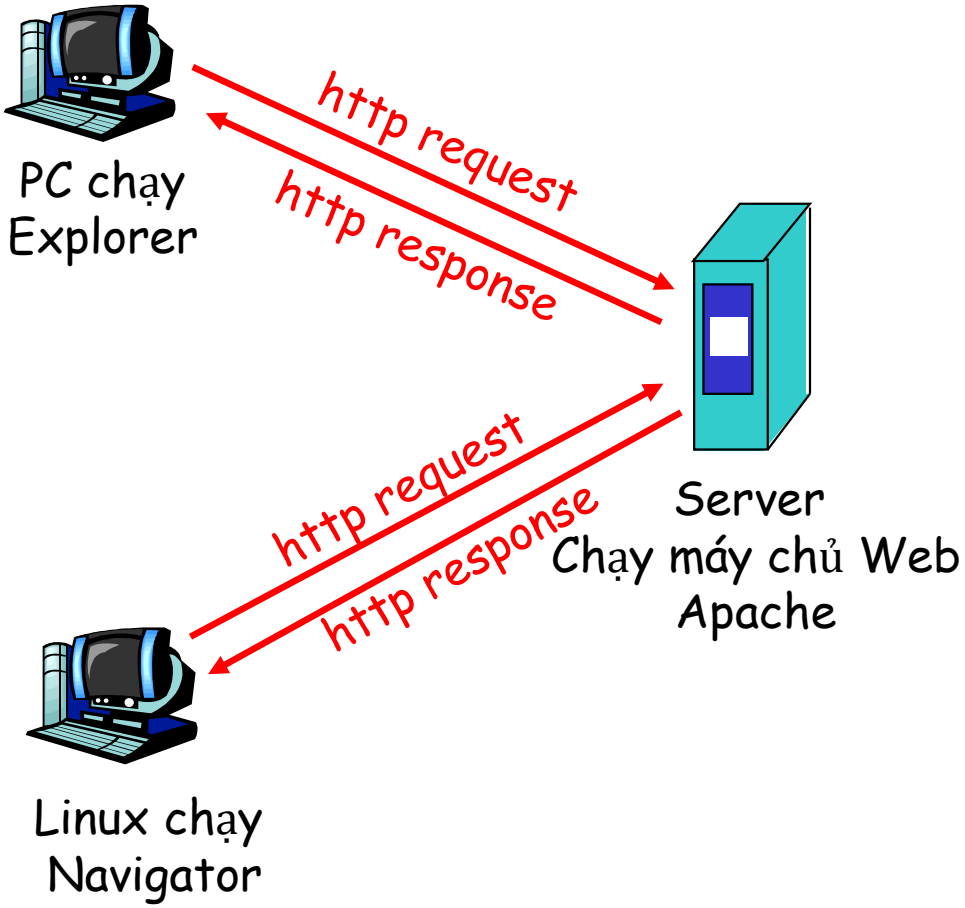
- Trang web:
  - Gồm các đối tượng.
  - Chỉ định bởi URL
  - Dựa vào trang HTML
  - Và một số tham chiếu đối tượng.
- URL có hai thành phần: tên miền, số hiệu cổng và đường dẫn
- User agent cho web là trình duyệt web (browser), e.g.
  - Mozilla Firefox
  - MS Internet Explorer
- Server cho web gọi là Web server:
  - Apache
  - MS Internet Information Server

http://it.tdtu.edu.vn:80/index.php

Giao thức Web-HTTP

HTTP: hypertext transfer protocol

- HTTP sử dụng TCP là dịch vụ vận chuyển.
- Mô hình client/server
  - *client:* trình duyệt web trình bày các đối tượng Web.
  - *server:* Web server gởi các đối tượng web cho client.
- http1.0: RFC 1945
- http1.1: RFC 2068



Luồng xử lý thông điệp HTTP 1.0

- Client thiết lập kết nối TCP (tạo ra socket) tới server, cổng 80
- Server đợi các yêu cầu từ client
- Client gởi yều tài liệu
- Web server gởi trả lại tài liệu tương ứng
- Đóng kết nối TCP

- Client phân tích tài liệu và nhúng các đối tượng vào (images)
  - Lặp lại cho các đối tượng khác.

Luồng xử lý thông điệp HTTP 1.0

Giả sử user nhập URL
it.tdtu.edu.vn/index.php

0. http server tại máy chủ it.tdtu.edu.vn đang đợi kết nối TCP tại cổng 80.

1a. http client thiết lập kết nối TCP tới http server tại it.tdtu.edu.vn. Cổng 80 là cổng mặt định cho http server.

1b. Server chấp nhận kết nối và gởi ack. cho client

2. http client gởi http *thông điệp yêu cầu* (chứa URL) vào cổng kết nối TCP Socket

3. http server nhận thông điệp yêu cầu, tạo ra thông điệp đáp trả chứa thông tin yêu cầu (index.html), và gởi vào socket.

time

Luồng xử lý thông điệp HTTP 1.0

4. http server đóng kết nối TCP.

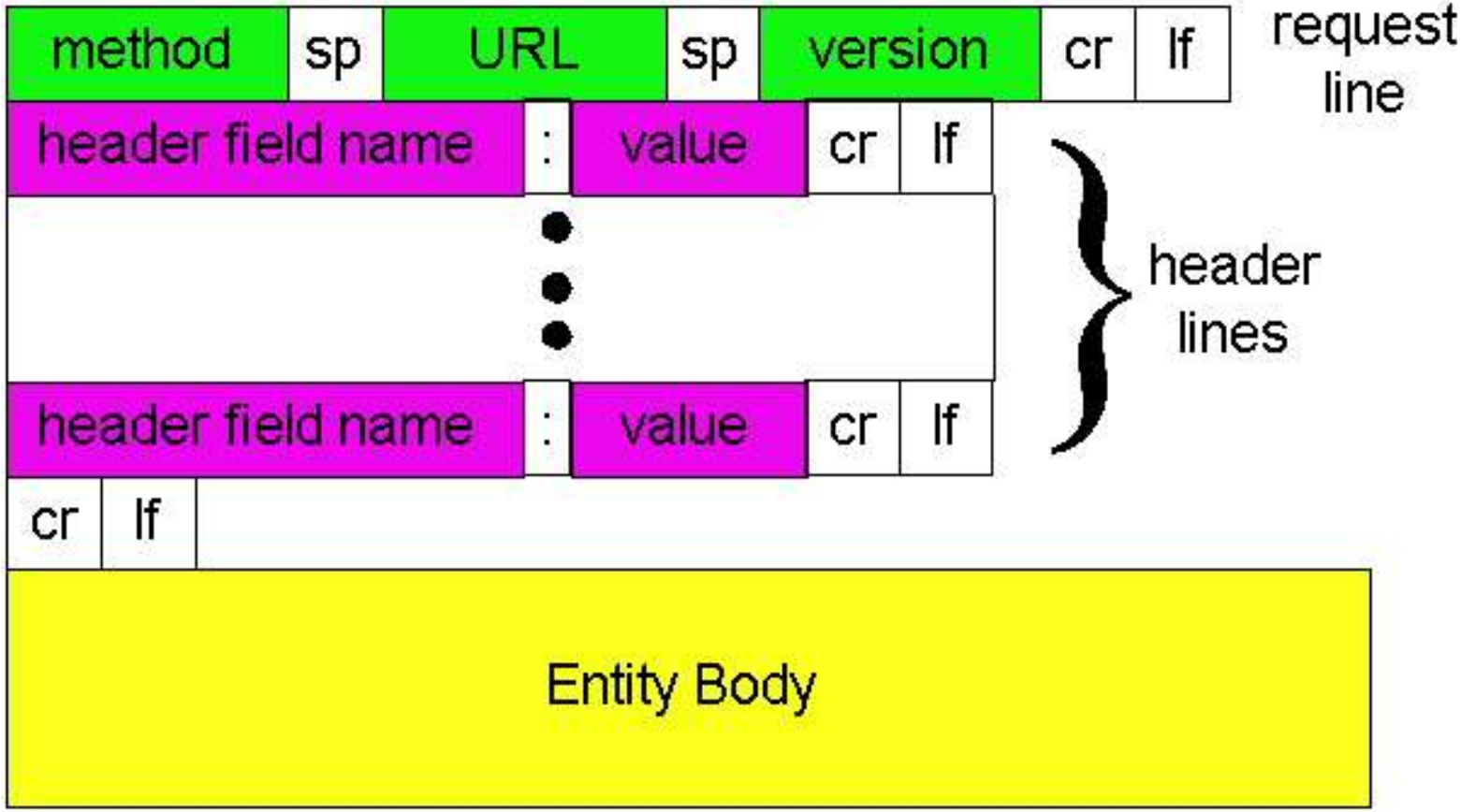
5. http client nhận thông điệp chứa tập tin html, phân tích tập tin này và nhúng các ảnh vào.

time

6. Các bước 1-5 lặp cho mỗi ảnh.

Khuôn dạng tổng quát của thông điệp yêu cầu HTTP

- ASCII (khuôn dạng có thể đọc được)



Ví dụ thông điệp yêu cầu: GET

Dòng yêu cầu (các lệnh GET, POST, HEAD)

Các dòng thông tin header

Các thông tin dữ liệu nếu có

```
GET /somedir/page.html HTTP/1.0
Host: www.somechool.edu
Connection: close
User-agent: Mozilla/4.0
Accept: text/html, image/gif, image/jpeg
Accept-language: fr
```

(extra carriage return, line feed)

Thông điệp đáp trả HTTP

Dòng trạng thái
(giao thức
Mã trạng thái
Trạng thái)

Các dòng thông tin
header

```
HTTP/1.0 200 OK
Date: Wed, 23 Jan 2008 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 …...
Content-Length: 6821
Content-Type: text/html

data data data data data ...
```

Dữ liệu, ví dụ,
Tập tin
Html được yêu cầu

Các mã trạng thái HTTP

Dòng đầu tiên server->client, một vài mã ví dụ

2xx OK

- Thành công

301 Moved Permanently

- Tài nguyên yêu cầu được chuyển sang vị trí mới

400 Bad Request

- Yêu cầu sai

404 Not Found

- Tài liệu không tìm thấy ở server

505 HTTP Version Not Supported

502 Dịch vụ quá tải

Kiểm tra HTTP

1. Telnet tới một Web server:

```
telnet it.tdtu.edu.vn 80
```

Mở kết nối TCP tới cổng 80
Mặt định là cổng 80

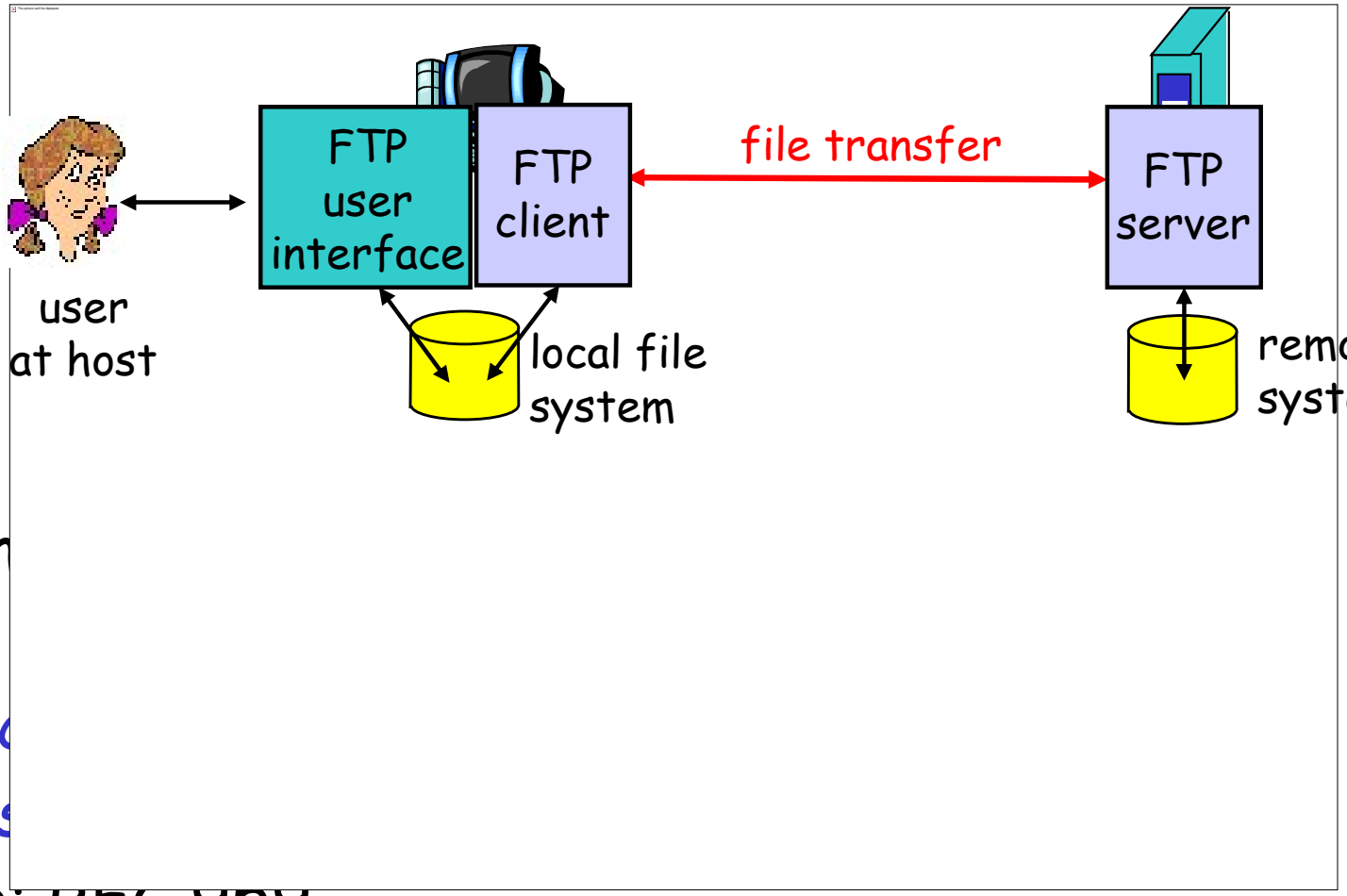
2. Gõ yêu cầu http GET:

```
GET /index.php HTTP/1.0
```

Đây là một phương thức để gởi yêu cầu thông điệp dạng GET HTTP tới Server.

3. Quan sát kết quả trả lại từ http server!

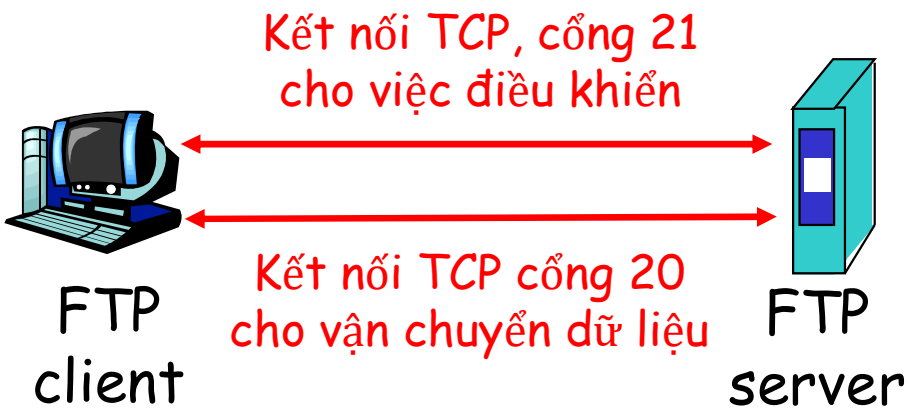
FTP: File Transfer Protocol



- Vận
- Mô
  - c
  - s
- ftp: RFC 959
- ftp server: port 21 (smtp 25, http 80)

FTP: Ứng dụng Client-Server sử dụng tách biệt kết nối điều khiển và kết nối dữ liệu

- ftp client gởi các lệnh tới ftp server tại cổng 21
- Mở hai kết nối song song
  - Điều khiển: trao đổi các lệnh, và các đáp trả giữa client và server.
  - Dữ liệu: dữ liệu là tập tin tới/từ server
- ftp server duy trì trạng thái kết nối, ví dụ,
  - Thư mục hiện hành,
  - Chứng thực trước đó



Các lệnh và thông điệp đáp trả FTP

Các lệnh mẫu:

- Lệnh và thông điệp ở dạng văn bản mã ASCII
- **USER *username***
- **PASS *password***
- **HELP** liệt kê các lệnh
- **LIST** trả lại danh sách tập tin trong thư mục hiện hành.
- **RETR filename** lấy tập tin có tên filename

Sample return codes

- Mã trạng thái và thông tin trạng thái (giống http)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can't open data connection**
- **452 Error writing file**

Thảo luận: Tại sao sử dụng khai kênh kết nối tách biệt dữ liệu và điều khiển?

Bài tập

- Tìm hiểu Firewall
- Tìm hiểu Proxy Server
- Tìm hiểu các phần mềm nguồn mở viết bằng java liên quan đến Firewall và Proxy Server trên mạng, cụ thể website sourceforge.net
- Viết báo cáo về chương trình (Lấy điểm, miễn thi)

Tổng quan lập trình java

Nội dung: Ôn tập JAVA

- Các thành phần cơ bản của java
- Đối tượng trong java, đa hình
- Kế thừa, liên kết muộn
- Giao diện
- Xử lý Ngoại lệ

Bài tập

- Viết chương trình Java cho phép người dùng nhập vào một dãy số nguyên, yêu cầu thực hiện tính tổng của dãy số đó.

  Ví dụ: n= 123 → Tổng = 6

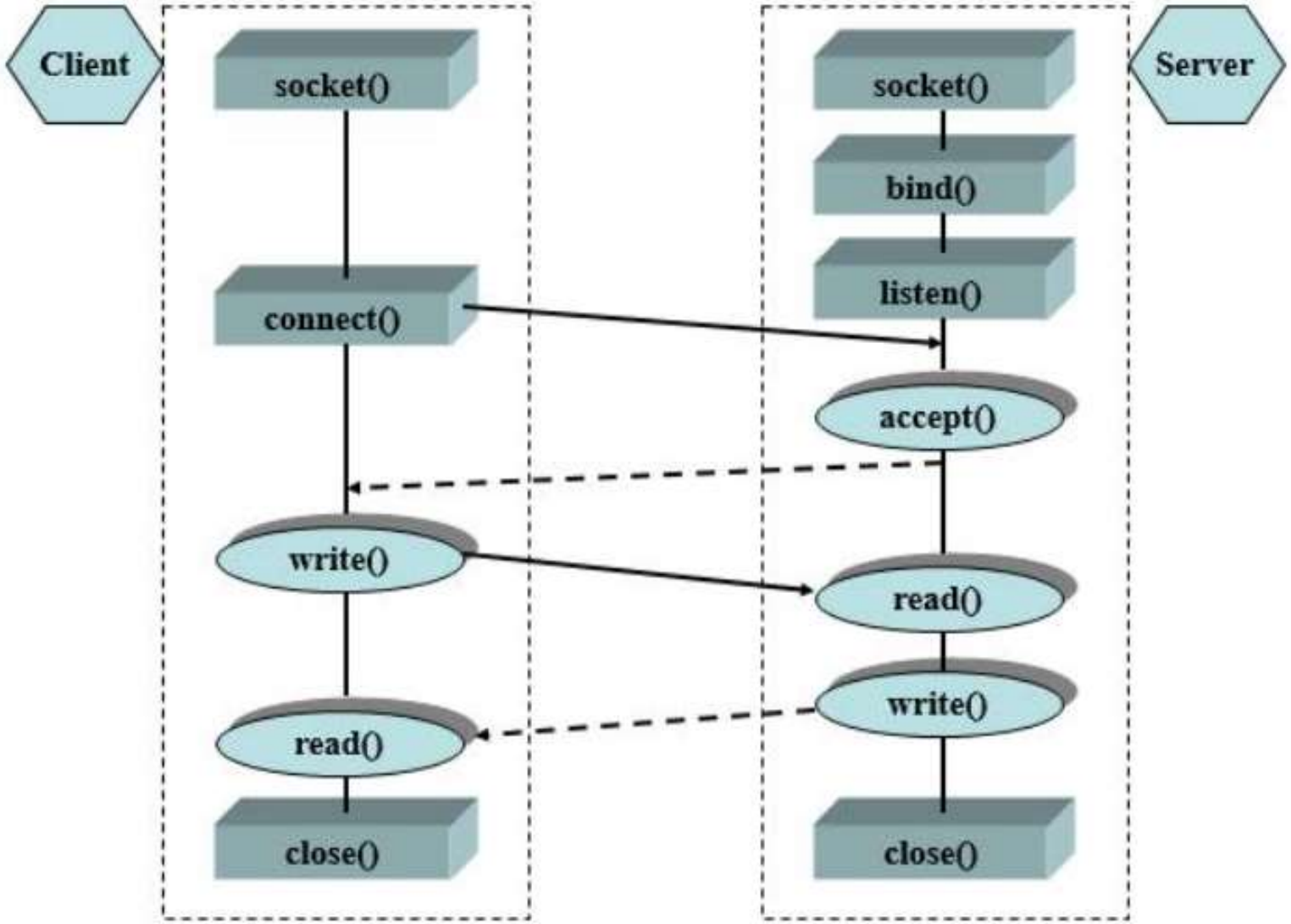
- Viết chương trình Java cho phép liệt kê các số nguyên tố nhỏ hơn số n cho trước? Sắp xếp dãy số theo thứ tự tăng dần?

TCP Socket

Nội dung

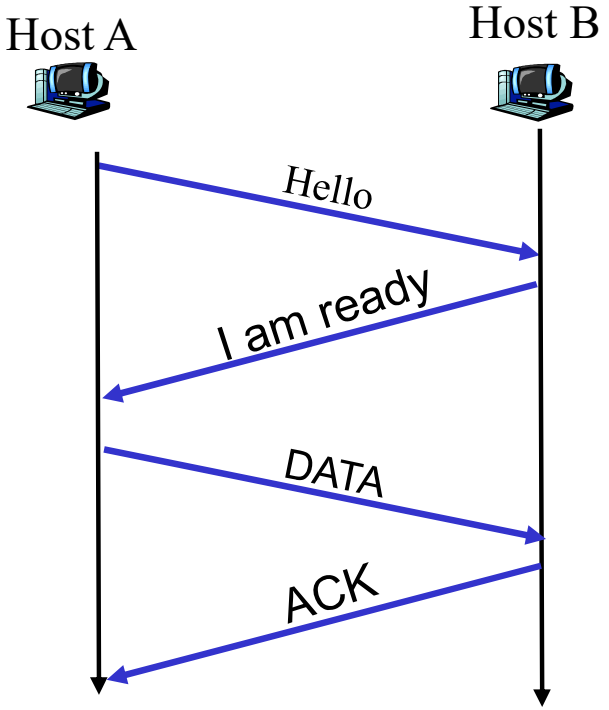
- Mô hình Client-Server
- Mô hình truyền tin Socket
- TCP Socket client - Socket
- TCP Socket server – ServerSocket

Nội dung


Client
socket()
connect()
write()
read()
close()
Server
socket()
bind()
listen()
accept()
read()
write()
close()

TCP

Source port			Destination port
Sequence Number			
Acknowledge Number			
Offset	Reserved	Flags	Window
Checksum			Urgent pointer
Options			Padding
Start of Data			



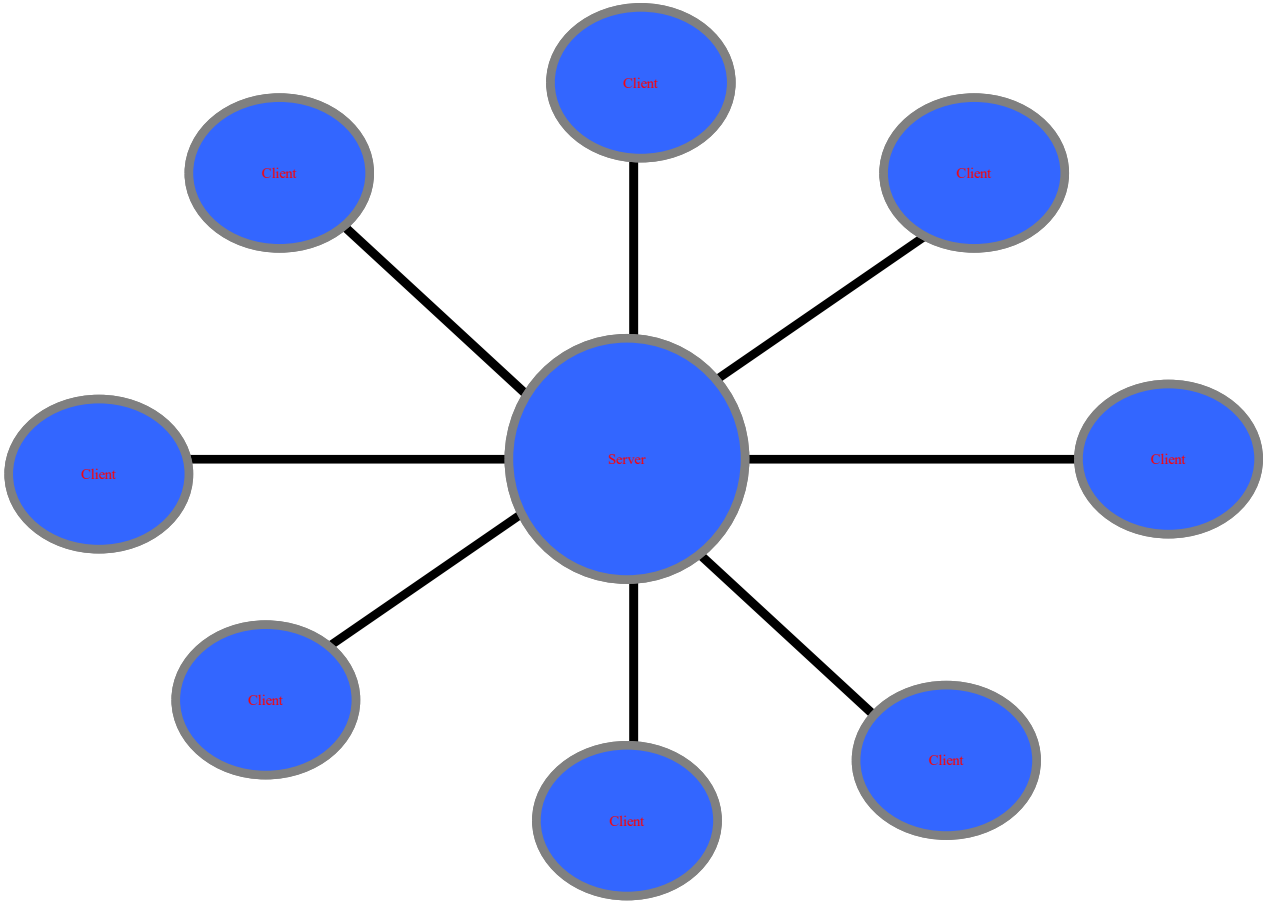
TCP

Trường	Mô tả
source port	Số hiệu cổng của nguồn
destination port	Số hiệu cổng đích
Sequence Number	Số thứ tự được tạo ra bởi nguồn
Acknowledge Number	Cho biết dữ liệu được nhận thành công.
Data offset	Các chi tiết về nơi dữ liệu gói tin bắt đầu
Reserved	Dự phòng
Flags	chỉ ra rằng gói tin cuối cùng hoặc gói khẩn cấp
Window	chỉ ra kích thước của vùng đệm nhận.
Checksum	xác định xem gói tin có bị hỏng không
Urgent Pointer	thông báo cho phía nhận biết có dữ liệu khẩn
Options	vùng dự phòng cho việc thiết lập trong tương lai
Padding	chỉ ra rằng dữ liệu kết thúc trong vòng 32 bit.

Số hiệu cổng TCP/UDP

- Có 3 loại (netstat –a)
  - số hiệu cổng hệ thống (0-1023). Các giao thức nổi tiếng có các số hiệu cổng nằm trong khoảng này.
  - Các số hiệu cổng người dùng (1024-49151). Các ứng dụng server của bạn sẽ nhận một trong các số này làm cổng, hoặc bạn có thể đăng ký số hiệu cổng với IANA .
  - Các số hiệu cổng riêng và động

Mô hình Client/Server



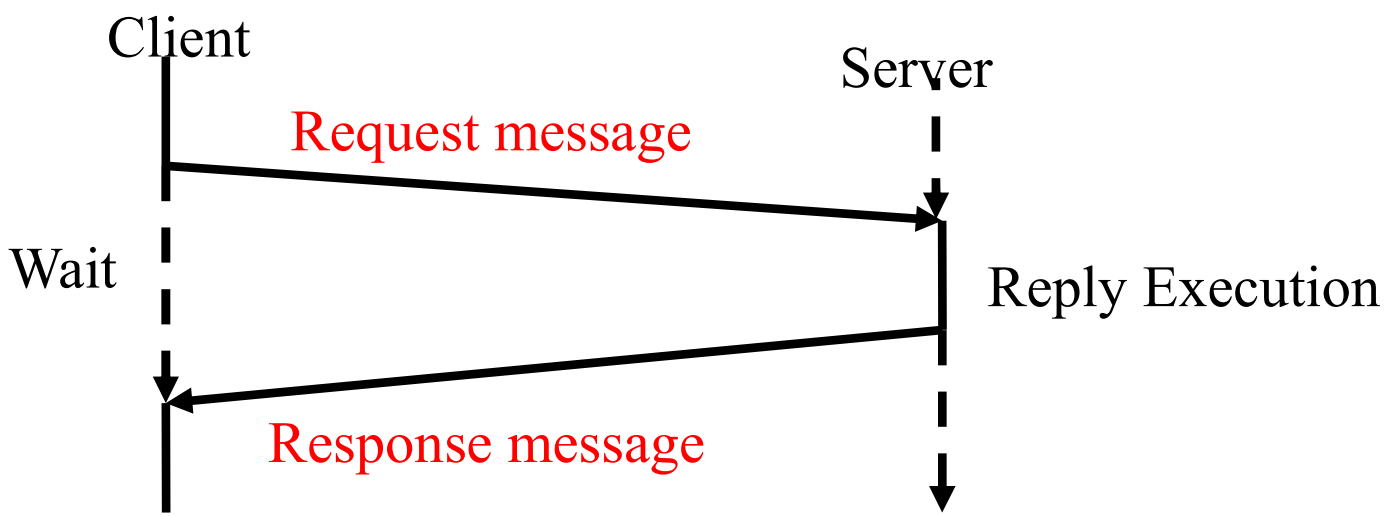
Mô hình Client/Server

Server

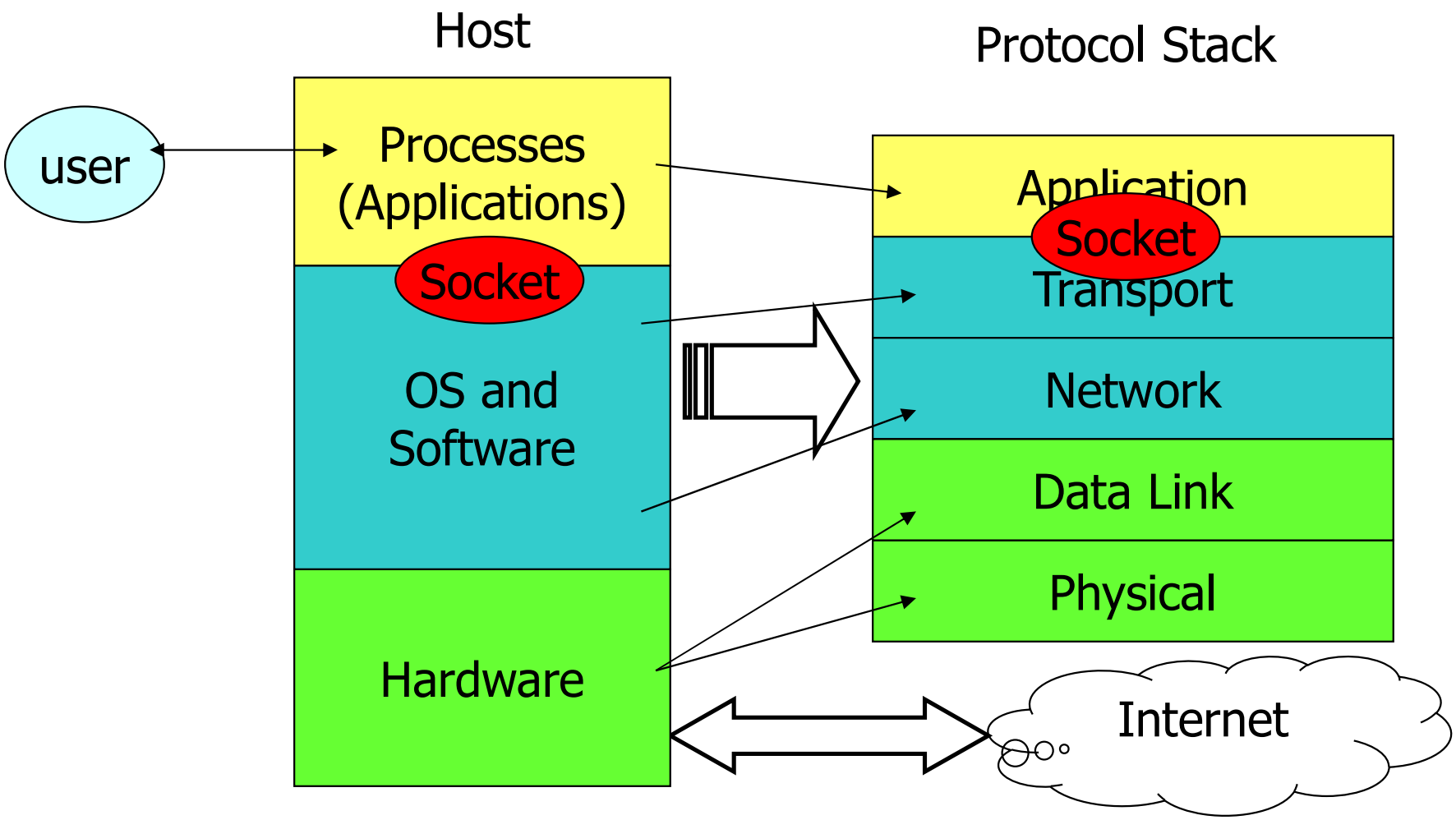
- Chạy trước
- Luôn đợi thiết lập kết nối từ client
- Đáp trả yêu cầu cho client tương ứng.

Client

- Chạy sau
- Thiết lập kết nối tới Server và gởi yêu cầu.
- Đợi đáp trả từ Server.



Mô hình truyền tin socket



Lớp Socket

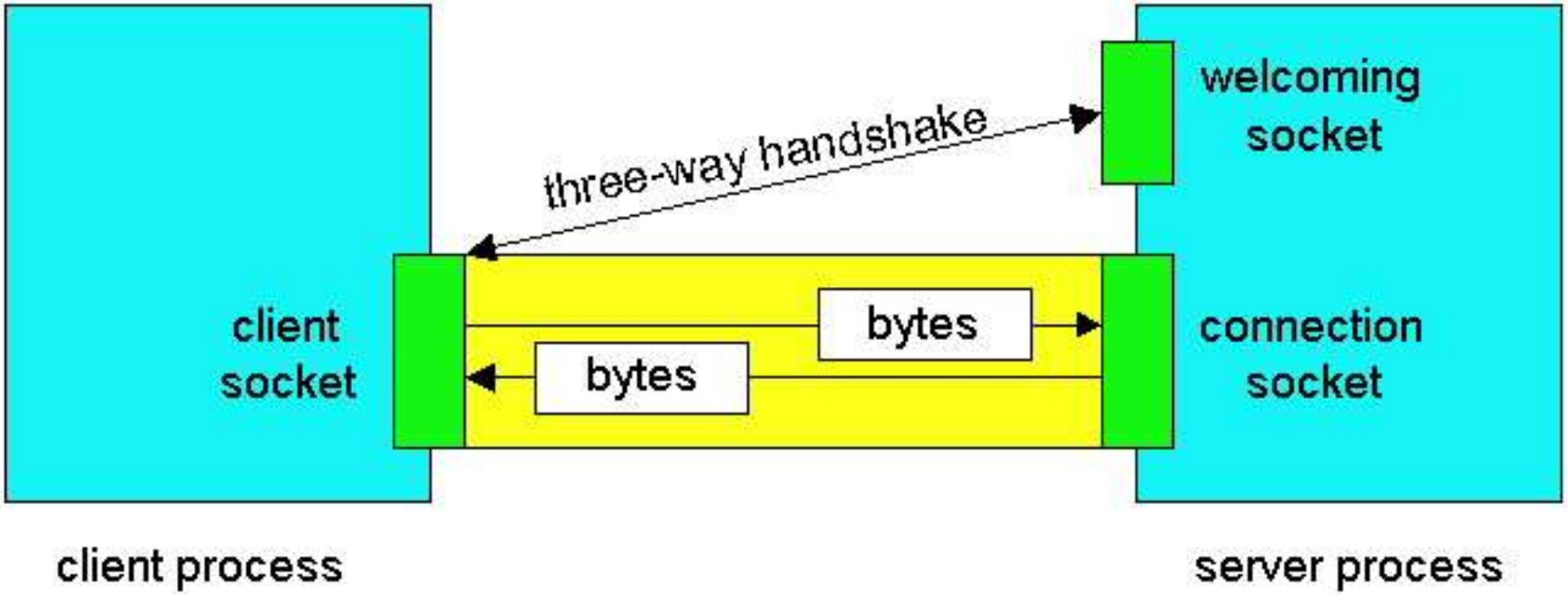
- Gói java.net
- TCP
  - Socket
  - ServerSocket
- UDP
  - DatagramPacket
  - DatagramSocket

TCP Socket

- java.net.Socket
  - Là đối tượng biểu diễn logic điểm đầu cuối của luồng giao tiếp giữa hai host.
  - Phương thức khởi tạo và các phương thức khác
    - Socket(String host, int port)
    - InputStream getInputStream()
    - OutputStream getOutputStream()
    - close()

- java.net.ServerSocket
  - Đợi kết nối từ client đến và tạo ra đối tượng socket để giao tiếp.
  - Constructor and Methods
    - ServerSocket(int port)
    - Socket Accept()

Socket


welcoming
socket
three-way handshake
client
socket
bytes
bytes
connection
socket
client process
server process

Phương thức lớp Socket

- Có 3 hàm khởi tạo
  - `Socket(InetAddress server, int port);`
  - `Socket(InetAddress server, int port, InetAddress local, int localport);`
  - `Socket(String hostname, int port);`
- Phương thức lớp Socket
  - `void close();`
  - `InetAddress getInetAddress();`
  - `InetAddress getLocalAddress();`
  - `InputStream getInputStream();`
  - `OutputStream getOutputStream();`

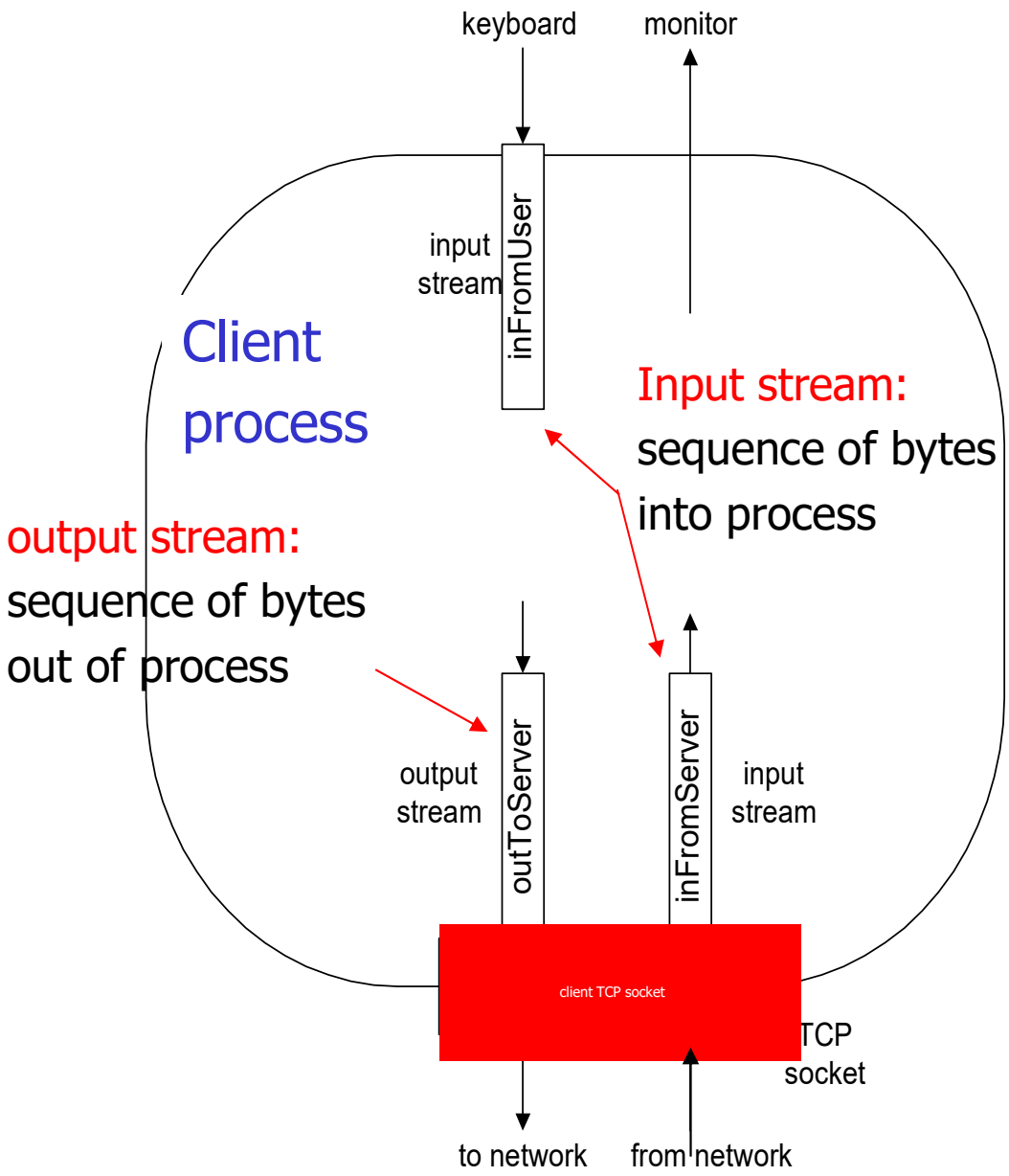
TCP Socket client

- Tạo đối tượng Socket kết nối tới Server
  - **Socket** s = new **Socket** ("java.sun.com", 8189)
- Lấy luồng gởi và đọc dữ liệu từ đối tượng Socket.
  - InputStream ins = s.**getInputStream** ();
  - OutputStream outs = s.**getOutputStream**();

Chú ý: "ins" và "outs" cùng kiểu với đối tượng "System.in" và "System.out" tương ứng.

- Gắn đối tượng đọc viết có định dạng vào luồng
  - BufferedReader in = new **BufferedReader**(new **inputStreamReader**(ins));
  - PrintWriter out = new **PrintWriter**(outs, true);
- Tiến hành đọc viết dữ liệu
  - String str = in.**readLine**();
  - out.**println** ("Echo:" + str + "\r");
- Đóng kết nối: s.**close**();

TCP Client


keyboard
monitor
input stream
inFromUser
Client process
Input stream: sequence of bytes into process
output stream: sequence of bytes out of process
output stream
outToServer
inFromServer
input stream
client TCP socket
TCP socket
to network
from network

Ví dụ TCP Client

- RFC867 - Daytime Protocol http://www.faqs.org/rfcs/rfc867.html

```
String hostname;
if (args.length > 0)   hostname = args[0];
else  hostname = "time.nist.gov";
try {
  Socket theSocket = new Socket(hostname, 13);
  InputStream timeStream = theSocket.getInputStream();
  StringBuffer time = new StringBuffer();
  int c;
  while ((c = timeStream.read()) != -1) time.append((char)c);
  String timeString = time.toString().trim();
  System.out.println("It is "+timeString+" at "+hostname);
}catch (UnknownHostException ex) {
     System.err.println(ex);
}catch (IOException ex) {System.err.println(ex); }
```

TCP Socket Server

- Tạo đối tượng Socket Server với một cổng lắng nghe xác định

```
SocketServer s = new SocketServer(8189)
```

- Đợi lắng nghe yêu cầu/kết nối từ client và tạo ra Socket giao tiếp.

```
While(true){
    Socket new_s = s.accept();
    doSomething(new_s);
}
```

- Xử lý giao tiếp với Socket client dựa vào đối tượng new_s này sử dụng luồng nhập/xuất.
- Đóng kết nối giao tiếp với client : `s.close();`

Ví dụ TCP Server

```
class TimeSvr {
    public static void main(String[] args ) {
        ServerSocket s = new ServerSocket(11113);
        while (true) {
            Socket new_s = s.accept();
            PrintWriter out = new
            PrintWriter(new_s.getOutputStream(), true);
            out.println(new Date());
            new_s.close();
        }
    }
}
```

Ví dụ TCP Client-Server

- RFC862 - Echo Protocol http://www.faqs.org/rfcs/rfc862.html

```
if (args.length > 0) {      hostname = args[0];    }
PrintWriter out = null;
BufferedReader networkIn = null;
try {
     Socket theSocket = new Socket(hostname, 7);
     networkIn = new BufferedReader( new InputStreamReader(theSocket.getInputStream()));
    BufferedReader userIn = new BufferedReader(new InputStreamReader(System.in));
     out = new PrintWriter(theSocket.getOutputStream());
    System.out.println("Connected to echo server");
     while (true) {
                String theLine = userIn.readLine();
                if (theLine.equals(".")) break;
                 out.println(theLine);
                 out.flush();
                System.out.println(networkIn.readLine());
      }
 } catch (IOException ex)  {      System.err.println(ex);    }
 finally {
       try {        if (networkIn != null) networkIn.close();  if (out != null) out.close();        }
        catch (IOException ex) {}
 }
```

TCP Server

```
class EchoSvr {
      public static void main(String[] args ) {
      String rdata;
      try {
            ServerSocket s = new ServerSocket(7);
            Socket con = s.accept();
            BufferedReader in;
           in = new BufferedReader(new
    InputStreamReader(con.getInputStream()));
            PrintWriter out = new PrintWriter(con.getOutputStream(), true);

           while ((rdata = in.readLine())!= null) {
                  System.out.println(rdata);
                  out.println(rdata);
            }
      } catch (Exception e) {  System.out.println(e);}
}
```

TCPClient.java

```
import java.io.*;
import java.net.*;

class TCPClient {
    public static void main(String argv[]) throws Exception
    {
        String sentence;
        String modifiedSentence;

        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));

        Socket clientSocket = new Socket("hostname", 6789);

        DataOutputStream outToServer =
                    new
    DataOutputStream(clientSocket.getOutputStream());
```

TCPClient.java

```
BufferedReader inFromServer =
            new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));

    sentence = inFromUser.readLine();

    outToServer.writeBytes(sentence + '\n');

    modifiedSentence = inFromServer.readLine();

    System.out.println("FROM SERVER: " + modifiedSentence);

    clientSocket.close();

    }
}
```

TCPServer.java

```
import java.io.*;
import java.net.*;
class TCPServer {
    public static void main(String argv[]) throws Exception
        {
            String clientSentence;
            String capitalizedSentence;

            ServerSocket welcomeSocket = new ServerSocket(6789);

            while(true) {

                Socket connectionSocket = welcomeSocket.accept();

                BufferedReader inFromClient = new BufferedReader(new
                        InputStreamReader(connectionSocket.getInputStream()));
```

TCPServer.java

```
    DataOutputStream  outToClient;
    outToClient = new
    DataOutputStream(connectionSocket.getOutputStream());

    clientSentence = inFromClient.readLine();

    capitalizedSentence = clientSentence.toUpperCase() + '\n';

    outToClient.writeBytes(capitalizedSentence);

      }
    }
}
```